



Pashov Audit Group

Olla Security Review

April 8th 2026 - April 21st 2026



Contents

1. About Pashov Audit Group	4
2. Disclaimer	4
3. Risk Classification	4
4. Methodology	5
5. About Olla	5
6. Executive Summary	5
7. Centralization & Trust	7
8. Findings	8
High findings	12
[H-01] Underfunded withdrawal requests can lead to cascading issues	12
[H-02] Withdrawal rate and exchange rate formula mismatch bias the slashing gate	14
[H-03] Stale <code>_accountingState.claimableRewards</code> in rebalance inflates <code>totalAssets</code>	16
Medium findings	18
[M-01] Unclaimed rewards on non-canonical rollups are excluded from accounting	18
[M-02] Slashing of existing attesters is not reflected before finalization	19
[M-03] Exchange rate staleness during slashing allows asset extraction via redemption	21
[M-04] Slashing adjustment clamping inflates gross reward	22
[M-05] RequestRedeem bypasses queue breaker causing liquidity issues and deposit blocks	23
[M-06] User-facing vault operations rely on stale <code>_accountingState</code> without on-chain update	24
[M-07] Stale <code>stakeRemaining</code> on <code>rebalance</code> resume skips recomputation of withdrawal	25
[M-08] Rate-drop breaker allows slow-bleed slashing to evade detection each cycle	26
[M-09] Stuck <code>FinalizeWithdrawals</code> step can lead to cascading issue	27
[M-10] Finalize <code>OllaVault::finalizeWithdrawals</code> with inadequate gas from outer loop	29
[M-11] <code>_refreshSingleAttester</code> updates aggregate state without notifying OllaCore	30
[M-12] Early return in <code>_refreshSingleAttester</code> misses slashing of a queued attester	31
[M-13] Invalid-key refund double counts in <code>totalAssets</code> inflating exchange rate	32
[M-14] <code>unstake()</code> loop is susceptible to OOG	34
[M-15] Governance transfer grants <code>DEFAULT_ADMIN_ROLE</code> to external wallet directly	35
[M-16] Exchange rate is temporarily low due to slashing event and withdrawal delay	36
[M-17] Accounting corruption in <code>processUnstakeAttester</code> enables exchange rate theft	37
[M-18] Queued attesters that become Aztec zombie exits cannot be recovered	38
Low findings	41
[L-01] Failed refunds from queue-entry are not accounted for until unrelated rebalance runs	41
[L-02] <code>refreshAttesterState</code> can finalize exit before GSE withdrawal unlocks	42
[L-03] Instant redemption lacks slippage protection	44
[L-04] Defensive underflow clamping masks aggregate state desynchronization	45
[L-05] Mint function lacks slippage protection	45
[L-06] <code>renounceOwnership()</code> is not disabled allowing permanent loss of admin control	46
[L-07] <code>OllaVault.recoverStAztec</code> gated by <code>whenNotPaused</code> blocks emergency recovery	46
[L-08] <code>_refreshSingleAttester</code> does not return after <code>_removeAttester</code> call execution	46
[L-09] StakeSurplus leaves nonzero allowance on StakingManager after partial stake	47
[L-10] Non-upgradeable <code>ReentrancyGuard</code> used in upgradeable contract	47
[L-11] Fee-rate changes apply retroactively to accrued rewards	48
[L-12] <code>requestRedeem()</code> is missing ERC-20 approve authorization path	49



[L-13] Claimed field in <code>WithdrawalRequest</code> is dead storage that breaks ERC-7540 spec	49
[L-14] <code>requestRedeem()</code> and <code>requestRedeemWithPermit()</code> lack <code>isPaused</code> safety check	50
[L-15] Governance role propagation revokes from wrong address	51
[L-16] Stuck rebalance step 5 on buffer drain forces rebalance revert	51
[L-17] Fully slashed attester DoSes <code>rebalance</code> until permissionless refresh clears it	52
[L-18] Vault ownership drift permanently bricks <code>WithdrawalQueue</code> UUPS upgrades	53
[L-19] <code>setOperator</code> is gated by <code>whenNotPaused</code> which can create weakness	53
[L-20] <code>setSafetyModule</code> does not update <code>lastAccountingTimestamp</code> on new safety module	54
[L-21] <code>WithdrawalMinimum</code> is share denominated	55
[L-22] <code>getClaimableRewards()</code> returns full value when <code>isRewardsClaimable</code> is false	55
[L-23] Rebalance buffer math ignores <code>_pendingClaimAmount</code>	56
[L-24] <code>upgradeCore()</code> and <code>upgradeSatellite()</code> hardcode empty init data	58
[L-25] Stale exchange rate due to infrequent accounting update	58
[L-26] <code>FinalizeWithdrawals</code> fails to increment <code>finalizedCount</code> for zero value requests	59
[L-27] Queued attesters can be incorrectly purged after a rollup upgrade	60
[L-28] Admin role not renounced after initialization	61
[L-29] Fee asset and staking asset mismatch can corrupt reward accounting	62
[L-30] <code>Has</code> <code>hasRebalanceWorkAvailable</code> does not check for buffer deficit	63
[L-31] Dust donation to <code>RewardsAccumulator</code> bypasses cooldown and idle rebalance check	63
[L-32] Out-of-band reward claims desync rewards reporting	64
[L-33] <code>RedeemRequest</code> event emits assets instead of shares violating ERC-7540	65
[L-34] Externally finalized exits can be swept before updating <code>_pendingClaimAmount</code>	65
[L-35] <code>StakeSurplus</code> step skips <code>canStake</code> check on fresh entry	66
[L-36] <code>SafetyModule</code> breaker stack cannot detect slashing without <code>refreshAttesterState</code>	67
[L-37] Instant-redemption fee accretes into share price	69



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Olla

Olla is an ERC7540-compliant liquid staking protocol that enables users to deposit assets into a vault, which are then staked to Aztec rollup attesters via a staking manager with automated rebalancing. The protocol includes an async withdrawal queue with instant redemption paths, a safety module that triggers protocol pauses on anomalous conditions, and a governance contract managing upgrades and protocol parameters.

Users deposit AZTEC into `OllaVault` to receive yield-bearing stAztec shares. The protocol stakes deposited funds across multiple Aztec rollup attesters, harvests staking rewards, and auto-rebalances to meet withdrawal demand. Withdrawals follow two paths: instant redemption (pays a fee, requires vault buffer) or async queue (FIFO, slower but fee-free).

The system is composed of:

- `OllaVault` — user-facing deposit/redeem interface; holds AZTEC buffer; mints/burns stAztec shares; delegates pricing to OllaCore
- `OllaCore` — accounting engine; runs permissionless multi-step rebalance state machine (harvest rewards, finalize unstakes, initiate new stakes); computes global exchange rate; mints protocol fees
- `StakingManager` — delegates vault's AZTEC to Aztec rollup attesters via stake/unstake calls; tracks per-attester exit lifecycle; harvests sequencer rewards
- `WithdrawalQueue` — FIFO queue for async redemptions; applies slashing adjustments during finalization if attesters were penalized between request and claim
- `SafetyModule` — circuit breaker that pauses protocol on rate drops, high queue ratio, or stale accounting; immutable trust anchor with no upgrade path
- `OllaGovernance` — timelock (2-day minimum delay) controlling upgrades, parameter changes, and role grants; emergency pause/unpause executes instantly via separate guardian role

6. Executive Summary

A time-boxed security review of the `ollafinance/core` repository was done by Pashov Audit Group, during which `ast3ros`, `DemoreXTess`, `TejasWarambhe`, `trtrth`, `ValvesSecurity` engaged to review Olla. A total of 58 issues were uncovered.



Protocol Summary

Project Name	Olla
Protocol Type	ERC7540 liquid staking protocol
Timeline	April 8th 2026 - April 21st 2026

Review commit hash:

- [f9c8502805dea9205dfda423a2cf24f261102b92](#)
(ollafinance/core)

Fixes review commit hash:

- [5be8601d6cd3d2002d2b3c8e484437fe89c361e7](#)
(ollafinance/core)

Scope

OllaCore.sol RewardsAccumulator.sol IOllaCore.sol IRewardsAccumulator.sol
GovernanceLib.sol IOllaGovernance.sol OllaGovernance.sol
ISafetyModule.sol SafetyModule.sol RolesLib.sol StakingManager.sol
StakingProviderRegistry.sol IAztecRollup.sol IAztecRollupRegistry.sol
IStakingManager.sol IStakingProviderRegistry.sol AztecTypes.sol
BN254Lib.sol QueueLib.sol OllaVault.sol StAztec.sol WithdrawalQueue.sol
IOllaVault.sol IStAztec.sol IWithdrawalQueue.sol



7. Centralization & Trust

The protocol has 3 privileged roles:

- **OllaGovernance (timelock)** — holds `DEFAULT_ADMIN_ROLE` on all contracts; 2-day minimum delay on proposals controlling upgrades (`upgradeCore` , `upgradeSatellite`), fourteen parameter setters (`setCore` , `setStakingManager` , `setSafetyModule` , `setInstantRedemptionFeeBP` , `setMaxInstantRedemptionFeeBP` , `setMinRebalancingCooldown` , `setTreasuryFeeBP` , `setProviderFeeBP` , `setMinRateDropBps` , `setMaxQueueRatioBps` , `setMaxAccountingDelay` , `setDepositCap` , `setMinProposalDelay` , `setGovernanceAdmin`), and role grants; emergency pause/unpause (`emergencyPauseAll` , `emergencyUnpauseAll`) executes instantly without timelock via separate `governanceAdmin` role
- **GUARDIAN_ROLE** — pauses/unpauses OllaCore, OllaVault, SafetyModule instantly via `pause` / `unpause` ; resets rebalance state machine via `forceRebalanceReset` (does not modify accounting or touch funds); cannot change parameters or access user funds
- **STAKING_PROVIDER_ADMIN_ROLE** — adds attester keys to StakingProviderRegistry via `dripQueue` ; sets rewards recipient per provider via `setProviderRewardsRecipient` ; not subject to timelock; cannot access protocol funds or modify system parameters



8. Findings

Findings count

Severity	Amount
High	3
Medium	18
Low	37
Total findings	58

Summary of findings

ID	Title	Severity	Status
[H-01]	Underfunded withdrawal requests can lead to cascading issues	High	Resolved
[H-02]	Withdrawal rate and exchange rate formula mismatch bias the slashing gate	High	Resolved
[H-03]	Stale <code>_accountingState.claimableRewards</code> in rebalance inflates <code>totalAssets</code>	High	Resolved
[M-01]	Unclaimed rewards on non-canonical rollups are excluded from accounting	Medium	Resolved
[M-02]	Slashing of existing attestors is not reflected before finalization	Medium	Resolved
[M-03]	Exchange rate staleness during slashing allows asset extraction via redemption	Medium	Resolved
[M-04]	Slashing adjustment clamping inflates gross reward	Medium	Resolved
[M-05]	RequestRedeem bypasses queue breaker causing liquidity issues and deposit blocks	Medium	Resolved
[M-06]	User-facing vault operations rely on stale <code>_accountingState</code> without on-chain update	Medium	Resolved
[M-07]	Stale <code>stakeRemaining</code> on <code>rebalance</code> resume skips recomputation of withdrawal	Medium	Resolved
[M-08]	Rate-drop breaker allows slow-bleed slashing to evade detection each cycle	Medium	Resolved



ID	Title	Severity	Status
[M-09]	Stuck <code>FinalizeWithdrawals</code> step can lead to cascading issue	Medium	Resolved
[M-10]	Finalize <code>OllaVault::finalizeWithdrawals</code> with inadequate gas from outer loop	Medium	Resolved
[M-11]	<code>_refreshSingleAttester</code> updates aggregate state without notifying OllaCore	Medium	Resolved
[M-12]	Early return in <code>_refreshSingleAttester</code> misses slashing of a queued attester	Medium	Resolved
[M-13]	Invalid-key refund double counts in <code>totalAssets</code> inflating exchange rate	Medium	Resolved
[M-14]	<code>unstake()</code> loop is susceptible to OOG	Medium	Resolved
[M-15]	Governance transfer grants <code>DEFAULT_ADMIN_ROLE</code> to external wallet directly	Medium	Resolved
[M-16]	Exchange rate is temporarily low due to slashing event and withdrawal delay	Medium	Resolved
[M-17]	Accounting corruption in <code>processUnstakeAttester</code> enables exchange rate theft	Medium	Resolved
[M-18]	Queued attesters that become Aztec zombie exits cannot be recovered	Medium	Resolved
[L-01]	Failed refunds from queue-entry are not accounted for until unrelated rebalance runs	Low	Resolved
[L-02]	<code>refreshAttesterState</code> can finalize exit before GSE withdrawal unlocks	Low	Resolved
[L-03]	Instant redemption lacks slippage protection	Low	Resolved
[L-04]	Defensive underflow clamping masks aggregate state desynchronization	Low	Resolved
[L-05]	Mint function lacks slippage protection	Low	Resolved
[L-06]	<code>renounceOwnership()</code> is not disabled allowing permanent loss of admin control	Low	Resolved
[L-07]	<code>OllaVault.recoverStAztec</code> gated by <code>whenNotPaused</code> blocks emergency recovery	Low	Resolved



ID	Title	Severity	Status
[L-08]	<code>_refreshSingleAttester</code> does not return after <code>_removeAttester</code> call execution	Low	Resolved
[L-09]	StakeSurplus leaves nonzero allowance on StakingManager after partial stake	Low	Resolved
[L-10]	Non-upgradeable <code>ReentrancyGuard</code> used in upgradeable contract	Low	Resolved
[L-11]	Fee-rate changes apply retroactively to accrued rewards	Low	Resolved
[L-12]	<code>requestRedeem()</code> is missing ERC-20 approve authorization path	Low	Resolved
[L-13]	Claimed field in <code>WithdrawalRequest</code> is dead storage that breaks ERC-7540 spec	Low	Resolved
[L-14]	<code>requestRedeem()</code> and <code>requestRedeemWithPermit()</code> lack <code>isPaused</code> safety check	Low	Acknowledged
[L-15]	Governance role propagation revokes from wrong address	Low	Resolved
[L-16]	Stuck rebalance step 5 on buffer drain forces rebalance revert	Low	Resolved
[L-17]	Fully slashed attester DoSes <code>rebalance</code> until permissionless refresh clears it	Low	Resolved
[L-18]	Vault ownership drift permanently bricks <code>WithdrawalQueue</code> UUPS upgrades	Low	Resolved
[L-19]	<code>setOperator</code> is gated by <code>whenNotPaused</code> which can create weakness	Low	Resolved
[L-20]	<code>setSafetyModule</code> does not update <code>lastAccountingTimestamp</code> on new safety module	Low	Resolved
[L-21]	<code>WithdrawalMinimum</code> is share denominated	Low	Acknowledged
[L-22]	<code>getClaimableRewards()</code> returns full value when <code>isRewardsClaimable</code> is false	Low	Resolved
[L-23]	Rebalance buffer math ignores <code>_pendingClaimAmount</code>	Low	Resolved



ID	Title	Severity	Status
[L-24]	<code>upgradeCore()</code> and <code>upgradeSatellite()</code> hardcode empty init data	Low	Resolved
[L-25]	Stale exchange rate due to infrequent accounting update	Low	Resolved
[L-26]	<code>FinalizeWithdrawals</code> fails to increment <code>finalizedCount</code> for zero value requests	Low	Resolved
[L-27]	Queued attesters can be incorrectly purged after a rollup upgrade	Low	Resolved
[L-28]	Admin role not renounced after initialization	Low	Resolved
[L-29]	Fee asset and staking asset mismatch can corrupt reward accounting	Low	Resolved
[L-30]	<code>Has hasRebalanceWorkAvailable</code> does not check for buffer deficit	Low	Resolved
[L-31]	Dust donation to <code>RewardsAccumulator</code> bypasses cooldown and idle rebalance check	Low	Resolved
[L-32]	Out-of-band reward claims desync rewards reporting	Low	Resolved
[L-33]	<code>RedeemRequest</code> event emits assets instead of shares violating ERC-7540	Low	Resolved
[L-34]	Externally finalized exits can be swept before updating <code>_pendingClaimAmount</code>	Low	Resolved
[L-35]	<code>StakeSurplus</code> step skips <code>canStake</code> check on fresh entry	Low	Resolved
[L-36]	<code>SafetyModule</code> breaker stack cannot detect slashing without <code>refreshAttesterState</code>	Low	Acknowledged
[L-37]	Instant-redemption fee accretes into share price	Low	Resolved



High findings

[H-01] Underfunded withdrawal requests can lead to cascading issues

Severity

Impact: High

Likelihood: Medium

Description

The `WithdrawalQueue::finalizeWithdrawals()` is used to finalize the pending withdrawal requests stored in the `_requests` storage mapping. This function loops through each request, applying a slashing adjustment (if applicable), before finalizing the withdrawal.

There can be a case where slashing occurred, which will first adjust the request's expected assets, and if the available buffer is insufficient to cover the request, the `while` loop will break:

```
function finalizeWithdrawals(uint256 available, uint256 currentRate)
    external
    override
    onlyVault
    nonReentrant
    returns (uint256 used, uint256 finalizedCount, uint256 totalAdjusted)
{
    // . . .

    while (currentId < upperBound) {
        if (gasleft() < _gasThreshold) {
            break;
        }

        WithdrawalRequest storage request = _requests[currentId];
        if (!request.finalized) {
            uint256 assetsExpected = request.assetsExpected;

            // Adjust payout when slashing has reduced the exchange rate below the locked
            // rate.
            // The > 1 tolerance accounts for floor-rounding differences between the
            // gross rate
            // (computed on aggregate state) and per-request rates (computed on per-
            // request state).
            if (currentRate < request.rate && request.rate - currentRate >
                _SLASHING_RATE_TOLERANCE) {
                uint256 payout = (request.shares * currentRate) /
                _RATE_SCALE;
                if (payout < assetsExpected) {
                    uint256 adjustment = assetsExpected - payout;
```



```
        pendingAssets -= adjustment;
        totalAdjusted += adjustment;
        request.assetsExpected = payout;
        assetsExpected = payout;
        emit WithdrawalAdjusted(currentId, assetsExpected + adjustment,
assetsExpected);
    }
}

// Slashing reduced payout to zero; finalize without consuming liquidity
// so the queue advances and the vault invariant
// (finalizedAmount == 0) == (finalizedCount == 0) is preserved.
if (assetsExpected == 0) {
    pendingShares_ -= request.shares;
    request.finalized = true;
    emit WithdrawalFinalized(currentId, 0);
    ++currentId;
    continue;
}

// Breaks on first under-funded request; does not skip.
if (available < assetsExpected) {                <<@
    break;
}

available -= assetsExpected;
used += assetsExpected;
pendingAssets -= assetsExpected;
pendingShares_ -= request.shares;
request.finalized = true;
++finalizedCount;
emit WithdrawalFinalized(currentId, assetsExpected);
}

++currentId;
}
// . . .
}
```



However, during such a scenario, the request is not finalized, but the accounting written before breaking out of the `while` loop is never reverted back to the original state. Such a case can lead to three issues:

1. If a call to `finalizeWithdrawals()` fires the slashing branch `(currentRate < request.rate)` and breaks on `available < payout`, `request.assetsExpected` is written down to `payout` in storage. This request will stay unfinalized. If `currentRate` recovers before the next finalize call (e.g., rewards accrue, slashing event reversed), the slashing branch is skipped (or computes a higher payout that now exceeds the already-reduced `assetsExpected`, so no re-adjustment fires). The user claims the locked-in reduced amount, and the haircut becomes permanent even though the protocol-wide rate recovered. Similarly, if the rate reduces again, the user will get a worse rate, even though technically their request was finalized.
2. `totalPendingAssets` accounting drops early as the `pendingAssets -= adjustment` accounting persists. `totalPendingAssets` no longer represents "assets owed at locked rate" for still-unfinalized requests; it is reduced to the slashed projection for the head request even though that request remains pending. This affects:
3. The queue-ratio circuit breaker, which sees a smaller queued number than actually owed.
4. This affects all `pendingWithdrawalAssets` reads.
5. The `cumulativeSlashingAdjustments` accounting grows without matching finalization.
6. `WithdrawalAdjusted()` event is emitted, but the paired `WithdrawalFinalized()` event for the same `currentId` is not and may never arrive for that (adjusted-old → adjusted-new) pair if the rate dips further on the next call, producing a second `WithdrawalAdjusted()` for the same id with a new old that equals the previous new.

In a large-scale slash scenario where the slash amount approaches the total buffer, this vulnerability causes all queued withdrawers to suffer the maximum haircut even when the contract held sufficient liquidity to honor their requests in full prior to the slash accounting write.

Recommendations

It is recommended to move the `available < expectedPayout` check before any storage write or revert the last request's state before terminating the loop.

[H-02] Withdrawal rate and exchange rate formula mismatch bias the slashing gate

Severity

Impact: Medium

Likelihood: High



Description

The `OllaVault::_executeRedeemRequest()` stores the exchange rate for the request made by the user into `request.rate`. The exchange rate formula can be written down as:

```
exchangeRate = (totalAssets + V) * 1e18 / (supply + V)
              = (B - pending + V) * 1e18 / (supply + V)

// B = bufferedAssets + stakedPrincipal + rewardsAccumulatorBalance + claimableRewards (This
// is what _computeTotalAssets computes when called with pendingWithdrawals = 0)
// pending = Sum of all assetsExpected across unfinalized queue entries
// (WithdrawalQueue.totalPendingAssets).
// V = Virtual offset
// supply = stAztec.totalSupply()
```

Similarly, `_withdrawalRate()` is passed into the `OllaCore::finalizeWithdrawals()` during rebalance:

```
_withdrawalRate = (B + V) * 1e18 / (supply + pendingShares + V)
```

The intent of slashing is "if the protocol's current per-share backing is below what this user locked in, slashing has happened; reduce their payout proportionally."

However, the formulas shown above would disagree with the intentions of the protocol:

```
/// @dev Uses gross total assets (before subtracting pending withdrawals) and gross total
supply
/// (including shares burned for pending requests). This ensures the rate reflects
the true
/// backing per share for adjustment after slashing, matching the rate stored at
request time. <@

function _withdrawalRate(IOllaVault vaultRef) internal view returns (uint256) {
```

Dropping `V` for brevity, let us set the formulas equal:

```
Abbreviate P = pending, s = supply, q = pendingShares.

exchangeRate = (B - P) / s          (Formula 1, net)
_withdrawalRate = B / (s + q)      (Formula 2, gross)
(B - P)/s = B/(s+q)
⇒ (B - P) · (s + q) = B · s
⇒ B · q = P · (s + q)
⇒ B/(s + q) = P/q
⇒ _withdrawalRate = P/q
```

This proves that formula 1 is equal to formula 2 only when `withdrawalRate` equals `P/q`. In simple words, the two formulas agree if and only if every pending request was locked at the same rate, specifically, the gross average rate. As soon as the queue holds entries locked at different rates (e.g., one from before a reward accrual, one from after), `P/q` is just their weighted average — and `B/(s+q)` will generally sit somewhere between the minimum and maximum locked rates.



As a consequence, the slashing gate will fire without the slashing actually occurring. When the queue holds heterogeneous locked rates:

- `_withdrawalRate` = weighted-average rate across gross shares
- Request locked below the average: `request.rate ≤ _withdrawalRate` → gate skipped → full payout. Correct.
- Request locked above the average: `request.rate > _withdrawalRate` → gate fires → payout written down.

This issue will fire up on every `rebalance()` call as the protocol accrues rewards continuously and users request redemption at different times. The impact can be observed as:

1. Tail of the queue redeemers will receive less than their locked `assetsExpected`.
2. Remaining shareholders receive direct per share value jumps as the pool now holds confiscated amounts.
3. Confiscated amounts strand in `_bufferedAssets`, creating a share-price anomaly for the next depositor. They either pay an inflated rate or receive fewer shares than intended.

Proof of concept for the same is available in the following gist - [link](#)

Recommendations

It is recommended to record the gross rate at the request time so that both sides use the same formula.

[H-03] Stale `_accountingState.claimableRewards` in rebalance inflates `totalAssets`

Severity

Impact: High

Likelihood: Medium

Description

`_accountingState.claimableRewards` mirrors the validator-side value returned by `StakingManager.getClaimableRewards()`. The mirror is only written inside `_updateAccountingInternal()`, which has two callers:

- `rebalance()` at the Done step (`OllaCore.sol:477`) — writes `0` because Harvest has just drained the validator.
- The permissionless `updateAccounting()` entry point (`OllaCore.sol:492`) — writes whatever the validator currently has claimable.

`_harvestRewards()` itself never touches the mirror:



```
// OllaCore.sol:632-649
function _harvestRewards() internal returns (uint256 rewardsDelta) {
    _modules.stakingManager.harvestRewards();           // validator -> RewardsAccumulator
    rewardsDelta = rewardsAccumulatorRef.recordBalance();
    if (rewardsDelta != 0) {
        _accountingState.cumulativeRewards += rewardsDelta;
    }
    emit RewardsDelta(rewardsDelta);
    _pullRewardsAccumulatorFunds();                     // RewardsAccumulator -> vault buffer
    // _accountingState.claimableRewards is NEVER touched here
}
```

`_pullRewardsAccumulatorFunds()` zeros `rewardsAccumulatorBalance`, but leaves `_accountingState.claimableRewards` holding `X`. The bucket is only refreshed at the Done step's final `_updateAccountingInternal()` call.

Between Harvest and Done, `_computeTotalAssets()` therefore double-counts `X`:

```
// OllaCore.sol:1365-1378
uint256 total =
    bufferedAssets + buckets.stakedPrincipal + buckets.rewardsAccumulatorBalance +
    buckets.claimableRewards;
// ^ already includes X                                     ^ still
holds stale X
```

Trigger sequence

For the bug to occur, the following calls must take place in order:

1. `updateAccounting()` — primes `_accountingState.claimableRewards = X` with the rewards currently claimable on the validator. Permissionless; naturally invoked by any keeper or monitoring system that publishes fresh `totalAssets()` between rebalances.
2. `rebalance()` → `_harvestRewards()` — moves those same `X` tokens out of the validator into the vault buffer, but leaves the storage mirror at `X`.
3. Inflated reading is consumed by one of:
4. the same `rebalance()` call's `_finalizeWithdrawals()` step, OR
5. an external `OllaVault.deposit()` / `OllaVault.instantRedeem()` call when `rebalance()` has paused between Harvest and Done on the gas gate (`OllaCore.sol:363`).

Recommendations

Zero `_accountingState.claimableRewards` inside `_harvestRewards()` at the moment the tokens physically leave the validator, so the accounting state stays in sync with the token flow for the rest of the rebalance.



Medium findings

[M-01] Unclaimed rewards on non-canonical rollups are excluded from accounting

Severity

Impact: Medium

Likelihood: Medium

Description

`StakingManager` resolves the rollup exclusively through `rollupRegistry.getCanonicalRollup()`, so both `getClaimableRewards()` and `harvestRewards()` only ever see the current canonical instance:

```
function harvestRewards() external override onlyCore nonReentrant returns (uint256 harvested) {
>>>    (, IAztecRollup rollup) = _getRollup();
    try rollup.claimSequencerRewards(rewardsAccumulator) returns (uint256 claimed) {
        harvested = claimed;
    } catch (bytes memory reason) {
        emit RewardsHarvestFailed(reason);
        return 0;
    }
    emit RewardsHarvested(harvested);
    return harvested;
}
```

```
function getClaimableRewards() external view override onlyCore returns (uint256 claimableRewards) {
>>>    (, IAztecRollup rollup) = _getRollup();
    return rollup.getSequencerRewards(address(rewardsAccumulator));
}
```

However, Aztec stores `sequencerRewards` per rollup instance, and `claimSequencerRewards` only pays out the balance held by the rollup it is called on:

```
function claimSequencerRewards(address _sequencer) internal returns (uint256) {
    RewardStorage storage rewardStorage = getStorage();
    require(rewardStorage.isRewardsClaimable, Errors.Rollup__RewardsNotClaimable());

    RollupStore storage rollupStore = STFLib.getStorage();
>>>    uint256 amount = rewardStorage.sequencerRewards[_sequencer];

    if (amount > 0) {
        rewardStorage.sequencerRewards[_sequencer] = 0;
        rollupStore.config.feeAsset.safeTransfer(_sequencer, amount);
    }
}
```



```
    return amount;
}
```

When Governance adds a new rollup via `Registry.addRollup()`, which advances the canonical pointer, if the previous rollup still holds unclaimed rewards for `rewardsAccumulator` at that moment, `getClaimableRewards()` immediately stops reporting them and `harvestRewards()` can no longer retrieve them. `updateAccounting()` consequently understates `totalAssets` and the exchange rate until the legacy rollup is called with `claimSequencerRewards`.

During this stale window, deposits mint excess shares against the depressed rate while withdrawals and redemptions settle against an understated rate, producing an unfair value transfer from exiting users (and existing holders) to incoming depositors. Rewards already pulled into `RewardsAccumulator` are still counted; the blind spot is limited to balances left unclaimed on non-canonical rollups.

Recommendations

Olla should support tracking legacy rollups and include their claimable rewards in accounting until they are fully harvested.

[M-02] Slashing of existing attesters is not reflected before finalization

Severity

Impact: Medium

Likelihood: Medium

Description

When `_processUnstakeAttester` initiates an exit, it snapshots the exit amount into `info.pendingExitAmount` and adds the same value to `_aggregateState.pendingUnstakeAmount`:

```
function _processUnstakeAttester(IAztecRollup rollup, address attester, AttesterInfo
storage info)
    internal
    returns (uint256 exitAmount)
{
    ...
    _setAttesterStatus(attester, info, InternalAttesterStatus.Exiting);
    uint256 cachedStake = info.stakedAmount;
    info.stakedAmount = 0;
    info.exitRollup = address(rollup);
>>> info.pendingExitAmount = SafeCast.toUint96(exitAmount);
```



```
// Update running state: subtract the full cached balance (not just the exit amount)
// to avoid leaving a phantom when the attester was slashed before unstaking.
if (_aggregateState.stakedAmount >= cachedStake) {
    _aggregateState.stakedAmount -= cachedStake;
} else {
    _aggregateState.stakedAmount = 0;
}
>>> _aggregateState.pendingUnstakeAmount += exitAmount;

...
}
```

On the Aztec rollup, an exiting attester remains slashable until `exitableAt`, and a slash reduces `exit.amount` in place while the exit record still exists.

However, `_refreshSingleAttester` only reconciles this reduction when the exit becomes exitable or disappears. The intermediate case - exit still pending, already slashed — is not handled:

```
function _refreshSingleAttester(IAztecRollup canonicalRollup, address attester) internal
{
    ...
    if (info.status == InternalAttesterStatus.Exiting) {
        if (!view_.exit.exists) {
            ...
        } else if (_isExitExitable(view_)) {
            ...
        } // Missing: exit exists, not yet exitable, but view_.exit.amount <
        pendingExitAmount
    }

    ...
}
```

For the entire pre-finalization window after such a slash,

`_aggregateState.pendingUnstakeAmount` (and therefore `StakingManager.totalStaked()`) remains inflated by the slashed amount. Even if someone calls `refreshAttesterState` and then `updateAccounting`, the protocol still cannot reflect the exit-delay slashing until the exit becomes exitable or disappears.

Because OllaCore reads `totalStaked` as `stakedPrincipal` during `updateAccounting`, `totalAssets()` and the exchange rate stay overstated until the exit finalizes. Any deposit or instant redemption that occurs in this window is mispriced: depositors receive fewer shares than they should, and instant redeemers extract more assets than they should, with the loss socialized to remaining holders once the slash is eventually booked.

Recommendations

Update `_refreshSingleAttester()` to reconcile slashing for all `Exiting` attesters whenever `view_.exit.exists == true`, not only when the exit is exit-able.



[M-03] Exchange rate staleness during slashing allows asset extraction via redemption

Severity

Impact: Medium

Likelihood: Medium

Description

`OllaCore.totalAssets()` computes total protocol value from `_accountingState`, a cached snapshot that is only refreshed during `rebalance()` or `updateAccounting()`:

```
function totalAssets() public view override returns (uint256) {
    IOllaVault vaultRef = IOllaVault(_modules.vault);
    return _computeTotalAssets(
        _accountingState,
        vaultRef.bufferedAssets(),
        vaultRef.pendingWithdrawalAssets()
    );
}
```

When slashing occurs on the Aztec rollup, the actual staked value decreases, but `_accountingState.stakedPrincipal` retains the pre-slashing value until the next accounting update. During this staleness window, `instantRedeem()` pays out shares at the inflated pre-slashing exchange rate:

```
function _redeem(address owner, uint256 shares, address recipient, bool
sharesPulledToVault)
    internal
    returns (uint256 netAssets)
{
    // ...
    if (sm.isPaused()) revert OllaVault__SafetyModulePaused();
    // ...
    uint256 grossAssets = coreRef.convertToAssets(shares); // uses stale totalAssets()
    uint256 fee = grossAssets * instantRedemptionFeeBP / BP_DIVISOR;
    netAssets = grossAssets - fee;
    // ...
}
```

The SafetyModule circuit breakers (`checkAccountingLiveness`, `checkRateDrop`) cannot prevent this because they are only invoked inside `_updateAccountingInternal()` — the very function that has not been called yet. The `sm.isPaused()` check in `_redeem()` only returns `true` if the breaker was already triggered by a prior accounting update.



An attacker monitoring the rollup can detect a slashing event and call `instantRedeem()` before the keeper calls `updateAccounting()`, extracting assets at the stale rate. The loss is socialized to remaining shareholders when accounting is eventually updated. The extraction is bounded by `availableForInstantRedemption()` (buffer minus pending withdrawals), but this can be substantial in a well-capitalized protocol.

Concrete example with 100M TVL, 20M buffer, 10% slash on 80M staked:

- Pre-slashing `totalAssets` = 100M, post-slashing real value = 92M
- Attacker holds 10M stAztec (10% of supply)
- `instantRedeem()` at stale rate: `grossAssets` = 10M (instead of correct 9.2M)
- With 5% exit fee: `netAssets` = 9.5M (instead of correct 8.74M)
- Extraction profit: approximately 760K at the expense of remaining shareholders

Recommendations

Add an accounting liveness check to `_redeem()` so that instant redemptions are blocked when accounting is stale beyond the configured threshold:

[M-04] Slashing adjustment clamping inflates gross reward

Severity

Impact: High

Likelihood: Low

Description

In `_computeNetFlows()`, the slashing adjustment subtraction from `netWithdrawals` uses unsigned arithmetic with a clamp-to-zero pattern:

```
netWithdrawals = netWithdrawals > adjustmentsSinceReport
    ? netWithdrawals - adjustmentsSinceReport
    : 0;
```

When `adjustmentsSinceReport` exceeds `netWithdrawals` (which occurs when slashing adjustments in the withdrawal queue exceed actual user withdrawals in a given reporting period), the excess is silently dropped. This makes `netWithdrawals` zero instead of allowing it to go negative, which in turn inflates `netFlows`:

```
netFlows = SafeCast.toInt256(netDeposits) - SafeCast.toInt256(netWithdrawals);
```

The inflated `netFlows` then propagates into `_computeGrossRewards()`:

```
grossRewardsSigned = changeInAssets - netFlows;
```



Because `netFlows` is artificially high (less was subtracted than should have been), `grossRewardsSigned` becomes artificially large. This inflated `grossRewards` value is used to compute protocol fees via `_payoutOllaProtocolFees()`, which mints excess shares to the treasury and staking provider, diluting all other shareholders.

The magnitude of the inflation equals the dropped excess: `adjustmentsSinceReport - netWithdrawals`. This scenario is realistic when a large slashing event occurs during a period with low withdrawal activity.

Over multiple reporting periods, each period's slash delta is silently zeroed, causing a compounding overstatement of `totalAssets()`. An attacker monitoring the accounting state can redeem shares after several slash events to extract value proportional to the accumulated phantom yield.

Recommendations

Replace the unsigned clamping with signed arithmetic to preserve the full value:

```
int256 adjustedWithdrawals = int256(netWithdrawals) - int256(adjustmentsSinceReport);  
netFlows = int256(netDeposits) - adjustedWithdrawals;
```

This allows the slashing adjustment excess to flow through correctly as a negative contribution to `netFlows`, preventing phantom reward inflation.

[M-05] RequestRedeem bypasses queue breaker causing liquidity issues and deposit blocks

Severity

Impact: Medium

Likelihood: Medium

Description

When pending withdrawals exceed `maxQueueRatioBps` (50%), `SafetyModule.checkQueueRatio` trips the breaker and sets `paused = true`. Deposits and instant redemptions check this flag `sm.isPaused()` and revert. But `requestRedeem` never checks it. It only uses the vault's separate OZ `whenNotPaused` modifier, which reads a flag that the `SafetyModule` never touches.

The result is that the breaker's response works against its own purpose. It blocks deposits, the one thing that would bring fresh liquidity and dilute the ratio. It leaves `requestRedeem` wide open, the one thing that grows the queue and makes the ratio worse.



No attacker is needed. A normal liquidity crunch is enough. Elevated withdrawal demand pushes the ratio past the threshold, the breaker fires, deposits stop flowing in, and panicking users who see the pause pile into `requestRedeem` because it still works. The queue grows, the ratio stays high or climbs further, and the guardian has to manually unpause into a situation that immediately re-trips.

Rebalance still runs (Core's OZ pause is independent from the SafetyModule flag) and initiates unstakes, but the Aztec exit delay is around 2 days. Until those exits complete and enough buffer accumulates to finalize the FIFO head, the protocol oscillates between paused and briefly unpaused with deposits blocked during every paused window.

Recommendations:

Make the breaker response match the breaker reason. Allow deposits during QueueRatio pauses, since new deposits dilute the ratio and accelerate recovery. Block `requestRedeem` during QueueRatio pauses, since new queue entries worsen the exact condition the breaker fired on.

[M-06] User-facing vault operations rely on stale `_accountingState` without on-chain update

Severity

Impact: Medium

Likelihood: Medium

Description

Every user-facing vault operation - `_deposit`, `mint`, `_redeem`, `_executeRedeemRequest` - prices shares through `OllaCore.convertToShares()` / `convertToAssets()`, which derive from `totalAssets()`. That function reads three fields from the cached `_accountingState` struct (`stakedPrincipal`, `rewardsAccumulatorBalance`, `claimableRewards`) that are only refreshed when a keeper calls `updateAccounting()` or completes a full `rebalance()` cycle. Between those calls, the on-chain exchange rate silently diverges from reality.

`SafetyModule.checkAccountingLiveness()` exists as a staleness circuit breaker, but it only executes on keeper paths - at the top of `rebalance()` and inside `_updateAccountingInternal()`. None of the four user-facing entry points invoke it. The SafetyModule's `isPaused()` check in `_deposit` and `_redeem` would catch a breaker that has already fired, but between the moment accounting goes stale and the moment a keeper triggers the check, user transactions execute against a rate that does not reflect the current protocol state.

The staleness cuts both ways depending on what has changed:



After an unaccounted slash (`stakedPrincipal` is overstated):

- **Deposits / mints** receive fewer shares than fair value. `convertToShares(assets)` divides by an inflated `totalAssets`, so the depositor overpays. Their capital subsidizes existing holders.
- **Redeems / withdrawal requests** receive more assets than the shares are worth. `convertToAssets(shares)` multiplies by the inflated `totalAssets`, extracting value from remaining holders.

After unaccounted reward accrual (`rewardsAccumulatorBalance` and `claimableRewards` are understated):

- **Deposits / mints** receive more shares than fair value, diluting existing holders who earned those rewards.
- **Redeems / withdrawal requests** receive fewer assets than owed, leaving earned yield on the table.

Recommendations:

Call `checkAccountingLiveness()` at the top of `_deposit`, `mint`, `_redeem`, and `_executeRedeemRequest`. If accounting has been stale longer than `maxAccountingDelay`, the breaker fires and blocks the transaction instead of executing at a stale rate.

[M-07] Stale `stakeRemaining` on `rebalance` resume skips recomputation of withdrawal

Severity

Impact: Medium

Likelihood: Medium

Description

When `rebalance()` reaches the `StakeSurplus` step, it computes how much surplus buffer can be staked onto the rollup. If gas runs out before the stake executes, the computed `stakeRemaining` is persisted to storage and the function returns, expecting a follow-up call to finish the job.

On resume, the code sees a non-zero `stakeRemaining` and skips the fresh recomputation entirely. It goes straight to staking the full persisted amount. It never re-reads `pendingWithdrawalAssets` or `bufferedAssets`. Any `requestRedeem` calls that landed between the two transactions grew the withdrawal queue, but the stale `stakeRemaining` still reflects the old, smaller queue.



The resumed stake pulls the full stale amount from the vault buffer, which can now exceed the actual available surplus. The buffer drops below what the withdrawal queue needs, leaving pending requests unfunded until the next rebalance cycle initiates fresh unstakes and waits through the Aztec exit delay.

For example: the buffer holds 600k and there are no pending withdrawals, so `stakeRemaining` is computed as 600k. Gas runs out. Between transactions, a user requests redemption of 600k worth of shares. The fresh `requiredBuffer` is now 600k, but the resume path never checks. It stakes the full stale 600k. Buffer drops to zero. The entire withdrawal queue is unfunded.

1. Buffer = 600k, pendingWithdrawals = 0, targetBuffered = 0
2. Fresh stakeRemaining = 600k - (0 + 0) = 600k
3. Gas gate fires at line 431 → progress saved, function returns
4. Between tx1 and tx2: user calls requestRedeem(600k shares) → pendingWithdrawals now = 600k, but `_bufferedAssets` still = 600k
5. Resume: stakeRemaining = 600k (stale), canStake(600k) returns true (keys available)
6. `_stakeSurplus(600k)` → `transferToCore(600k)` succeeds (buffer = 600k ≥ 600k)
7. Buffer drops to 0. Withdrawal queue has 600k unfunded.

Recommendations:

Re-clamp `stakeRemaining` against the fresh surplus on resume. Before staking, recompute `requiredBuffer` from the current `pendingWithdrawalAssets` and `targetBufferedAssets`, then cap `stakeRemaining` at the actual safe surplus. If the safe surplus is zero, skip staking and move to Done.

[M-08] Rate-drop breaker allows slow-bleed slashing to evade detection each cycle

Severity

Impact: Medium

Likelihood: Medium

Description

`SafetyModule.checkRateDrop` compares the current exchange rate against the previous cycle's rate. The reference comes from `_latestReport.exchangeRate`, which is overwritten every time `_updateReportingSnapshots` runs during `rebalance()` or `updateAccounting()`. Each cycle's new, lower rate becomes the baseline for the next comparison.



```
function checkRateDrop(uint256 oldRate, uint256 nextRate) external override onlyCoreOrVault {  
    // solhint-disable-next-line gas-strict-inequalities  
    if (nextRate >= oldRate) {  
        return;  
    }  
  
    uint256 dropBps = (oldRate - nextRate) * BPS_DENOMINATOR / oldRate;  
    // solhint-disable-next-line gas-strict-inequalities  
    if (dropBps >= minRateDropBps) {  
        _triggerBreaker(ISafetyModule.BreakerReason.RateDrop);  
    }  
}
```

This means the breaker only catches single-cycle drops. A gradual degradation where each individual cycle stays below `minRateDropBps` accumulates without limit and never triggers the breaker. There is no cumulative tracking, no high-water mark, and no decay-resistant reference anywhere in the protocol.

With `minRateDropBps` set to 500 (5%), a single 6% drop in one cycle correctly trips the breaker. But the same 6% total loss spread across 10 cycles at 0.6% each never fires it. Each cycle re-anchors the reference, so the per-cycle drop is always below the threshold while the exchange rate quietly falls from 1.0 to 0.94. At the minimum cooldown of 10 minutes, the loss could be around 6% in under 2 hours without tripping the breaker. At the default 1-hour cooldown, it takes 10 hours. In both cases, the per-cycle 0.6% drop stays well below the 5% threshold while the cumulative loss is significant.

A validator experiencing gradual slashing penalties, a slow reward shortfall, or a sustained economic attack that degrades the rate by less than `minRateDropBps` per cycle can bleed value from the protocol indefinitely without any circuit breaker response. Users have no automated protection against slow-moving losses.

Recommendations

Track a high-water mark rate alongside the per-cycle reference. Compare against both: the per-cycle reference catches sudden drops, the high-water mark catches cumulative degradation. A separate `maxCumulativeDropBps` threshold with a governance-resettable high-water mark gives the protocol both fast and slow loss detection without false positives from legitimate rate movements such as fee dilution.

[M-09] Stuck `FinalizeWithdrawals` step can lead to cascading issue

Severity

Impact: Medium

Likelihood: Medium



Description

The `ollaCore::rebalance()` runs a state machine as follows:

Done → Harvest → PullUnstaked → FinalizeWithdrawals → InitiateUnstake → StakeSurplus → Done

To advance from `FinalizeWithdrawals` step, it requires any one of `pending == 0`, `buffer == 0`, or `finalizedAmount == 0`:

```
// slither-disable-next-line incorrect-equality,timestamp
if (progress.step == IOllaCore.RebalanceStep.FinalizeWithdrawals) {
    if (!_hasGasForStep()) {
        _rebalanceProgress = progress;
        return (rewardsDelta, 0, 0, vaultRef.bufferedAssets());
    }
    finalizedAmount = _finalizeWithdrawals();
    uint256 pending = vaultRef.pendingWithdrawalAssets();
    // slither-disable-next-line incorrect-equality,timestamp
    if (pending == 0 || vaultRef.bufferedAssets() == 0 || finalizedAmount == 0)
{
    <<@
        progress.step = IOllaCore.RebalanceStep.InitiateUnstake;
    } else {
        _rebalanceProgress = progress;
        return (rewardsDelta, finalizedAmount, 0, vaultRef.bufferedAssets());
    }
}
```

However, an attacker can forcefully ensure that each of these is non-zero; this can be done by ensuring that between each `rebalance()`, the attacker front-runs with:

1. `deposit(X)`: bumps `bufferedAssets()` back up (buffer was partially drained by the prior finalization).
2. `requestRedeem(100 shares)` — bumps `pendingWithdrawalAssets()` back up.

Next `rebalance()` runs `_finalizeWithdrawals()` again → finalizes the fresh request → `finalizedAmount > 0`, and post-finalize `pending > 0` && `buffer > 0` → stuck again.

This attack can be considered cheap as it only requires the rotation of `100 stAztec` and the `requestRedeem()` path has no fees. The impact of the attack can be observed as:

1. Accounting staleness can lead to a full protocol pause.

This is the most severe consequence, `_updateAccountingInternal()` only runs when `rebalance` reaches `Done`, which is the only code path that updates `lastAccountingTimestamp` in the `SafetyModule`. Every `rebalance()` call starts with `checkAccountingLiveness()`. If `block.timestamp - lastAccountingTimestamp > maxAccountingDelay` (range: 1 hour to 7 days), the circuit breaker fires and pauses the entire `SafetyModule`.

1. Governance is deadlocked



Many governance setters have the `whenRebalanceDone` modifier, including `setTargetBufferedAssets`, `setRebalanceCooldown`, etc. While the state machine is stuck mid-flight, governance cannot adjust parameters to mitigate the attack.

1. Fee distribution stalled

Protocol fees (treasury + provider splits) are only computed and minted inside `_updateAccountingInternal()` → `_computeAndFinalizeAccounting()` → `_payoutOllaProtocolFees()`. While stuck, no fees are distributed.

1. Yield Loss on Idle Buffer

Buffer earns zero yield. Only staked assets earn sequencer rewards. While stuck at `FinalizeWithdrawals`, the `StakeSurplus` step never executes, so all buffer sits idle. New deposits from legitimate users also sit idle, and they feed the attack by keeping `bufferedAssets() > 0`, preventing the escape condition.

Recommendations

Add a snapshot field to `RebalanceProgress` and track how much has been finalized against that snapshot. Advance once the snapshot is drained, regardless of new arrivals.

[M-10] Finalize `OllaVault::finalizeWithdrawals` with inadequate gas from outer loop

Severity

Impact: Medium

Likelihood: Medium

Description

The `WithdrawalQueue::finalizeWithdrawals()` uses an internal gas guard (`gasleft() < _gasThreshold`, configured to 80,000 in production) to bound its own loop.

The `OllaVault::finalizeWithdrawals` runs a second loop of the same length after the queue returns, which updates `_claimableShares` for every newly finalized request. That outer loop is O(N) in the batch size chosen by the inner loop but shares only the leftover gas margin (~80,000) the inner guard left behind.

```
// Inner guard bounds only its own work
while (currentId < upperBound) {
    if (gasleft() < _gasThreshold) {
        break;
    }
    ...
}
```



When enough requests with distinct recipients are pending, the outer loop performs an $O(N)$ update of `_claimableShares` at ~22-26k gas per first-time recipient (and ~5-8k per warm recipient). At realistic per-step budgets, the 80k guard never fires, the inner loop runs to completion, and the outer loop then OOGs on its own work. Because the revert rolls back the queue's inner writes, the next attempt finalizes the same batch and OOGs again. The redemption pipeline is deadlocked until governance (timelock-gated) calls `setQueueGasThreshold()`, which can make this DoS for around 2 days.

The attack/organic worst case is the cold-recipient path: new liquid-staking users arrive continuously, and fresh EOAs are free for an attacker to spin up and submit `requestRedeem()` from.

```
// Outer loop is unbounded by the queue's guard:
uint256 newPending = queue.nextUnfinalized();
for (uint256 id = prevPending; id < newPending; ++id) {
    IWithdrawalQueue.WithdrawalRequest memory req = queue.getRequest(id);
    if (req.finalized) {
        _claimableShares[_requestOwners[id]] += req.shares;
    }
}
```

At the 180k `rebalanceGasThreshold` entry gate and a conservative 1M per-step budget, any batch of ≥ 34 distinct-recipient withdrawals deadlocks the finalize flow. For organic LST load, 34 unique claimers per finalize window is well within the expected range; for an adversary with cheap `requestRedeem()`, it is trivially reachable.

Recommendations

It is recommended to drop the `gasleft()` heuristic and bound the work by an explicit `maxBatch` parameter on `WithdrawalQueue::finalizeWithdrawals()`. Fold the `_claimableShares[owner] += req.shares` update into the queue loop (move `_requestOwners()` / `_claimableShares()` into the queue, or have the queue accept a callback that the vault wires to its own state). Either fix eliminates the $O(N)$ post-processing on the vault side; together, they remove the coupling-based failure mode entirely.

[M-11] `_refreshSingleAttester` updates aggregate state without notifying OllaCore

Severity

Impact: Medium

Likelihood: Medium



Description

`refreshAttesterState` is a permissionless function that can update `_aggregateState.stakedAmount`, `_aggregateState.slashingDelta`, and `_aggregateState.pendingUnstakeAmount` for any number of attesters. OllaCore's accounting (exchange rate, `totalAssets()`, `_accountingState`) is only updated when `rebalance()` or `updateAccounting()` is explicitly called. Between a `refreshAttesterState` call that records a slashing event and the next OllaCore accounting update, `_accountingState.slashingDelta` and `stakedPrincipal` in OllaCore are stale, but `StakingManager.totalStaked()` already reflects the post-slash value. This inconsistency exists for the entire window between the refresh and the next OllaCore accounting call.

Impact

Any party that reads OllaCore's `totalAssets()` or the exchange rate during the staleness window sees a value that does not reflect the slashing already recorded in StakingManager. Depositors and withdrawers are priced against an inflated rate. The window length is unbounded; it depends entirely on how frequently keepers call `updateAccounting()` and can be extended by a keeper who deliberately delays the accounting update after submitting a large `refreshAttesterState` transaction that records a slashing loss.

Recommendations

OllaCore should be notified either via a callback or by having `refreshAttesterState` be callable only through OllaCore — whenever `_aggregateState` changes materially (slashing delta increase or significant balance decrease).

[M-12] Early return in `_refreshSingleAttester` misses slashing of a queued attester

Severity

Impact: High

Likelihood: Low

Description

When an attester has `InternalAttesterStatus.Queued`, `_refreshSingleAttester` checks whether the rollup has promoted it to `VALIDATING` and, if not, unconditionally returns early. This skips all slashing and exit-detection logic that follows. A Queued attester can be slashed between the time its deposit is submitted and the time `refreshAttesterState` is first called. In that window, the rollup may have already reduced or zeroed the attester's



`effectiveBalance` , or created an exit record, both of which are invisible to the early return. The `_aggregateState.stakedAmount` and `_aggregateState.slashingDelta` are never corrected for this loss.

Impact

If a Queued attester is slashed before its first refresh, the slashing loss is silently swallowed. `_aggregateState.stakedAmount` remains inflated by the full staked amount, `slashingDelta` is never incremented, and `totalStaked()` overstates the protocol's real collateral. This feeds into OllaCore's `totalAssets()` and the exchange rate, causing stAztec shares to be priced above their true backing. Depositors who enter during this window receive fewer shares than they should; existing holders are diluted when the discrepancy is eventually reconciled.

Recommendations

Remove the unconditional early return for Queued attesters that were not yet activated. Instead, fall through to the balance-delta and exit-detection logic so that a balance decrease or existing exit record is handled even while the attester's internal status is still `Queued` .

[M-13] Invalid-key refund double counts in `totalAssets` inflating exchange rate

Severity

Impact: High

Likelihood: Low

Description

When a staking provider admin enqueues a keystore with an invalid proof-of-possession via `StakingProviderRegistry.addKeysToProvider()` , the BLS key is not validated on-chain. During rebalance step 5 (`StakeSurplus`), `StakingManager._stakeAttesters()` calls `_setQueued(attester, X)` which increments `_aggregateState.stakedAmount += X` , then calls `rollup.deposit()` which only enqueues the entry without validating the PoP.

When the permissionless `rollup.flushEntryQueue()` is called, the invalid PoP causes the GSE deposit to fail, and the rollup refunds X tokens back to the `StakingManager` . No Olla-side callback fires, so `_aggregateState.stakedAmount` still includes X.

On the next rebalance, `_pullUnstakedFunds()` calls `getUnstakedFunds()` which sweeps the entire `StakingManager` balance (including the refunded X) to Core, which forwards it to Vault:



```
// StakingManager.getUnstakedFunds()
function getUnstakedFunds() external override onlyCore nonReentrant
    returns (uint256 received, uint256 exitAmount)
{
    exitAmount = _pendingClaimAmount; // 0 - refund never incremented this
    _pendingClaimAmount = 0;
    uint256 balance = stakingAsset.balanceOf(address(this)); // X (refunded)
    if (balance > 0) {
        stakingAsset.safeTransfer(core, balance);
    }
    received = balance;
}
```

In `_pullUnstakedFunds()`, because `exitAmount` = 0, the `stakedPrincipal` decrement is skipped:

```
// OllaCore._pullUnstakedFunds()
if (exitAmount > 0) { // SKIPPED - exitAmount is 0
    _accountingState.stakedPrincipal -= exitAmount;
}
```

At the end of rebalance, `_updateAccountingInternal()` reads `stakingManager.totalStaked()` which still includes X in `_aggregateState.stakedAmount`, and overwrites `_accountingState.stakedPrincipal` with this inflated value:

```
// totalStaked() still includes X
stakedPrincipal = modules.stakingManager.totalStaked();
// Overwrites cached stakedPrincipal with value that includes X
_applyAccountingUpdates(stakedPrincipal, ...);
```

The result is that X is double-counted in `totalAssets()`: once in `_bufferedAssets` (from the sweep) and once in `stakedPrincipal` (from `totalStaked()`). This inflates the exchange rate, allowing the attacker to redeem shares at an inflated rate and extract value from remaining shareholders.

`purgeFailedQueueEntry()` only corrects `_aggregateState.stakedAmount` in `StakingManager` — it does not trigger a Core accounting update. The double count persists until the next `rebalance()` or `updateAccounting()` call, giving the attacker a window to redeem.

Recommendations

The cleanest fix is to make `getUnstakedFunds()` distinguish between exit-related funds and unexpected surplus (refunds).



[M-14] `unstake()` loop is susceptible to OOG

Severity

Impact: High

Likelihood: Low

Description

The `StakingManager::unstake` iterates `_activeAttesterSet` from the end, calling `_processUnstakeAttester`, which makes two external rollup calls per active attester until the requested `amount` is satisfied:

```
// src/staking/StakingManager.sol:191-214
function unstake(uint256 amount) external override onlyCore nonReentrant returns (uint256
unstakedAmount) {
    if (amount == 0) revert StakingManager__ZeroAmount();
    (, IAztecRollup rollup) = _getRollup();

    uint256 length = _activeAttesterSet.length();
    if (length == 0 || _activeCount == 0) {
        return 0;
    }

    for (uint256 i = length; i > 0;) {                                <<@

        --i;
        if (unstakedAmount >= amount) break;

        address attester = _activeAttesterSet.at(i);
        AttesterInfo storage info = _attesterMap[attester];
        if (info.status != InternalAttesterStatus.Active) continue;

        uint256 exitAmount = _processUnstakeAttester(rollup, attester, info);    <<@
        unstakedAmount += exitAmount;
    }

    return unstakedAmount;
}
```

The only loop-exit conditions are (1) `amount` satisfied, or (2) `i == 0`. There are no gas checks, which are not in sync with how the sibling function `_stakeAttesters()` handles the same using gas threshold:

```
for (uint256 i; i < attestersToStakeTo; ++i) {
    if (gasleft() < _gasThreshold) {                                <<@
        break;
    }
    ...
}
```



Under mass redemption, the parent function `rebalance()` can fully revert, no progress is persisted, and every subsequent call reproduces the same OOG with the same `unstakeRemaining`.

Recommendations

It is recommended to add a similar fix matching the `_stakeAttesters()` pattern:

```
for (uint256 i = length; i > 0;) {
    --i;
    if (unstakedAmount >= amount) break;
+   if (gasleft() < _gasThreshold) break;

    address attester = _activeAttesterSet.at(i);
    AttesterInfo storage info = _attesterMap[attester];
    if (info.status != InternalAttesterStatus.Active) continue;

    uint256 exitAmount = _processUnstakeAttester(rollup, attester, info);
    unstakedAmount += exitAmount;
}
```

[M-15] Governance transfer grants `DEFAULT_ADMIN_ROLE` to external wallet directly

Severity

Impact: High

Likelihood: Low

Description

`OllaGovernance` is designed to be the sole `DEFAULT_ADMIN_ROLE` holder on all satellite contracts, with governance actions flowing through its timelock. This invariant breaks during governance transfer.

When `acceptGovernance` is called, `_propagateAdminRole` grants `DEFAULT_ADMIN_ROLE` on `OllaVault`, `WithdrawalQueue`, `RewardsAccumulator`, `StakingManager`, `StakingProviderRegistry`, and `SafetyModule` directly to the incoming governance wallet, an external EOA or contract operating outside the timelock.

```
function _propagateAdminRole(address oldGovernance, address newGovernance) internal {
    ...

    address[6] memory satellites = [vaultAddr, wq, rv, sm, spr, sfm];

>>> _propagateSatelliteAdminRoles(satellites, oldGovernance, newGovernance);
}
```



This creates two immediate timelock bypass paths:

- `SafetyModule`'s sensitive setters are protected only by `onlyRole(DEFAULT_ADMIN_ROLE)`, so the new governance wallet can directly change the deposit cap, withdrawal minimum, rate-drop threshold, queue ratio threshold, and accounting delay threshold without any timelock flow.
- `OllaVault` grants `DEFAULT_ADMIN_ROLE` to governance and `CORE_ROLE` to core at initialization. `transferToCore` is gated only by `CORE_ROLE`, but a direct `DEFAULT_ADMIN_ROLE` holder can self-grant `CORE_ROLE` and call `transferToCore` to pull buffered assets to an arbitrary address. A governance wallet rotation therefore becomes a potential fund drain path that bypasses the timelock's intended user exit window.

Recommendations

Do not propagate satellite `DEFAULT_ADMIN_ROLE` to the external governance wallet during `acceptGovernance()`. Governance transfer should only rotate the timelock operator roles; the satellite admin should remain on the `OllaGovernance` contract itself.

[M-16] Exchange rate is temporarily low due to slashing event and withdrawal delay

Severity

Impact: Medium

Likelihood: Medium

Description

`OllaCore._exchangeRate` computes the rate as $(totalAssets() + offset) / (totalSupply + offset)$, where `totalAssets` subtracts `pendingWithdrawalAssets` which are locked at the rate that prevailed at request time.

When a slash reduces `stakedPrincipal`, `totalAssets` drops immediately, but `pendingWithdrawalAssets` remains at its pre-slash value. This creates a double discount in the numerator: the slash loss is absorbed once through the reduced `stakedPrincipal`, and again through the stale (overstated) `pendingWithdrawalAssets` deduction.

```
uint256 total = bufferedAssets
               + buckets.stakedPrincipal // reduce immediately when slash
               + buckets.rewardsAccumulatorBalance
               + buckets.claimableRewards;

               - pendingWithdrawals // reduce in withdrawal finalization
```



During the window between the slash being reflected in accounting and withdrawal finalization, the exchange rate is more depressed than the economically correct rate for remaining holders. This mispricing has two consequences:

- Depositors receive more shares than fair value, which become extractable profit once the rate corrects upward at finalization.
- Instant redeemers receive fewer assets than fair value, with the loss redistributed to remaining holders.

Recommendations

`totalAssets` should subtract the slash-adjusted expected payout rather than the original locked value.

[M-17] Accounting corruption in `processUnstakeAttester` enables exchange rate theft

Severity

Impact: High

Likelihood: Low

Description

When an attester is slashed below the ejection threshold on the Aztec rollup, their `effectiveBalance` is set to 0 and the remaining safe funds are moved into an `Exit` struct with `isRecipient = false` (a "zombie exit").

```
function slash(address _attester, uint256 _amount) internal {
    ...
} else {
    ...
    if (isRemoved && toUser > 0) {
        // Only if we remove the attester AND there is something left will we create an exit
        store.exits[_attester] = Exit({
            withdrawalId: withdrawalId,
            amount: toUser,
            exitableAt: Timestamp.wrap(block.timestamp) + store.exitDelay.decompress(),
            recipientOrWithdrawer: withdrawer,
            isRecipient: false,
            exists: true
        });
    }
    ...
}
```

If `OllaCore` triggers `rebalance` and selects this attester for unstaking before `refreshAttesterState` is called, `_processUnstakeAttester` misreads the exit amount:



```
function _processUnstakeAttester(IAztecRollup rollup, address attester, AttesterInfo
storage info)
    internal
    returns (uint256 exitAmount)
    {
        AttesterView memory view_ = rollup.getAttesterView(attester);
>>> exitAmount = view_.effectiveBalance; // Returns 0 for a zombie-exited attester

        bool isInitiated = rollup.initiateWithdraw(attester, address(this));
        ...

        if (_aggregateState.stakedAmount >= cachedStake) {
>>> _aggregateState.stakedAmount -= cachedStake; // Full pre-slash balance deducted
        } else {
            _aggregateState.stakedAmount = 0;
        }
        _aggregateState.pendingUnstakeAmount += exitAmount;

        // Record the slashing gap so OllaCore's accounting sees it.
        if (cachedStake > exitAmount) {
            _aggregateState.slashingDelta += (cachedStake - exitAmount); // Entire
cachedStake recorded as slashing loss
        }

        ...
    }
}
```

Because `effectiveBalance` is 0 for a zombie-exited attester, the function subtracts the full pre-slash `cachedStake` from `stakedAmount` and adds the entire `cachedStake` to `slashingDelta`. The correct exit amount (`view_.exit.amount`) is never read.

When `updateAccounting` is subsequently called, `totalAssets` is deflated by the phantom slashing loss, artificially depressing the exchange rate. An attacker can exploit this predictable sequence, monitoring for slash events, triggering `rebalance` to induce the corrupted unstake, calling `updateAccounting` to lock in the deflated rate, and depositing at the reduced rate to profit when the rate recovers.

Recommendations

In `_processUnstakeAttester`, detect the zombie exit case and read `view_.exit.amount` directly instead of `view_.effectiveBalance`.

[M-18] Queued attesters that become Aztec zombie exits cannot be recovered

Severity

Impact: High

Likelihood: Low



Description

When `StakingManager` stakes a new attester, it records the attester locally as `Queued`, adds the full activation threshold to `_aggregateState.stakedAmount`, and deposits into the rollup with `address(this)` as the withdrawer.

```
function _setQueued(address attester, uint256 stakedAmount) internal {
    AttesterInfo storage info = _attesterMap[attester];
    if (info.attester != address(0)) {
        revert StakingManager__AttesterAlreadyActive(attester);
    }
    info.attester = attester;
    info.stakedAmount = stakedAmount;
    ++_attesterCount;
>>> _setAttesterStatus(attester, info, InternalAttesterStatus.Queued);
    _aggregateState.stakedAmount += stakedAmount;
}
```

If the rollup flushes its entry queue and the attester is then slashed on Aztec before `refreshAttesterState` promotes it to `Active`, and the slash leaves residual funds, Aztec creates an exit with `isRecipient = false` and reports the attester as `ZOMBIE`. Only the withdrawer, `StakingManager`, can convert that zombie exit into a claimable one via `initiateWithdraw`. However, none of `StakingManager`'s recovery paths handle the `Queued + ZOMBIE` state:

```
function getStatus(address _attester) internal view returns (Status) {
    Exit memory exit = getExit(_attester);
    uint256 effectiveBalance = getStorage().gse.effectiveBalanceOf(address(this), _attester);

    Status status;
    if (exit.exists) {
>>> status = exit.isRecipient ? Status.EXITING : Status.ZOMBIE;
    } else {
        status = effectiveBalance > 0 ? Status.VALIDATING : Status.NONE;
    }

    return status;
}
```

`_refreshSingleAttester` only promotes `Queued` attesters on `VALIDATING`; any other rollup status (including `ZOMBIE`) returns early.

```
function _refreshSingleAttester(IAztecRollup canonicalRollup, address attester) internal
{
    ...
    AttesterView memory view_ = rollup.getAttesterView(attester);

    // Handle Queued attesters: check if the rollup has activated them.
    if (info.status == InternalAttesterStatus.Queued) {
        if (view_.status == Status.VALIDATING && view_.effectiveBalance > 0) {
            // Attester has been activated on the rollup -- promote to Active.
            _setAttesterStatus(attester, info, InternalAttesterStatus.Active);
            emit AttesterStateRefreshed(attester, info.stakedAmount,
view_.effectiveBalance);
        }
    }
}
```



```
    }  
    // Still queued (NONE on rollup) -- nothing to reconcile yet.  
>>>    return;  
    }  
  
    ...  
}
```

`purgeFailedQueueEntry` requires the rollup state to be exactly `Status.NONE` with no exit and a zero balance, which excludes `ZOMBIE`.

```
function purgeFailedQueueEntry(address attester) external override nonReentrant {  
    ...  
    (, IAztecRollup rollup) = _getRollup();  
    AttesterView memory view_ = rollup.getAttesterView(attester);  
>>>    if (view_.status != Status.NONE || view_.exit.exists || view_.effectiveBalance > 0) {  
        revert StakingManager__NotFailedQueueEntry(attester);  
    }  
  
    ...  
}
```

`unstake` iterates only `_activeAttesterSet`, so queued attesters are never visited.

```
function unstake(uint256 amount) external override onlyCore nonReentrant returns  
(uint256 unstakedAmount) {  
    ...  
>>>    uint256 length = _activeAttesterSet.length();  
    if (length == 0 || _activeCount == 0) {  
        return 0;  
    }  
  
    // Iterate active attesters from the end to avoid index shifting on removal.  
>>>    for (uint256 i = length; i > 0;) {  
        ...  
    }  
  
    return unstakedAmount;  
}
```

The attester therefore remains stuck as `Queued` indefinitely. Its full stake stays counted in `_aggregateState.stakedAmount`, inflating the exchange rate denominator, and the residual funds in the zombie exit cannot be claimed through the normal flows.

Recommendations

Handle `Queued` attesters with `view_.exit.exists` or `Status.ZOMBIE` inside `_refreshSingleAttester()` instead of returning early.



Low findings

[L-01] Failed refunds from queue-entry are not accounted for until unrelated rebalance runs

Description

When a queued attester deposit later fails during the rollup's entry-queue flush, the rollup refunds the stake to the configured withdrawer, which in Olla's stake path is

`StakingManager` :

```
function flushEntryQueue(uint256 _toAdd) internal {
    ...
    if (success) {
        ...
    } else {
        require(data.length > 0, Errors.Staking__DepositOutOfGas());
    >>> store.stakingAsset.safeTransfer(args.withdrawer, amount);
        emit IStakingCore.FailedDeposit(
            args.attester, args.withdrawer, args.publicKeyInG1, args.publicKeyInG2,
args.proofOfPossession
        );
    }
}
...
}
```

The refunded tokens now sit as a raw balance on `StakingManager` . To reconcile, anyone can call `purgeFailedQueueEntry` , which subtracts the stale `cachedStake` from `_aggregateState.stakedAmount` and removes the attester:

```
function purgeFailedQueueEntry(address attester) external override nonReentrant {
    ...
    uint256 cachedStake = info.stakedAmount;
    if (_aggregateState.stakedAmount >= cachedStake) {
        _aggregateState.stakedAmount -= cachedStake;
    } else {
        _aggregateState.stakedAmount = 0;
    }

    _setAttesterStatus(attester, info, InternalAttesterStatus.Exiting);
    _removeAttester(attester);

    emit FailedQueueEntryPurged(attester, cachedStake);
}
```

Because the refunded amount is never credited to `_pendingClaimAmount` or any other tracked bucket, and since `totalStaked` sums only the three tracked fields, the refund drops out of `stakedPrincipal` and therefore out of `OllaCore.totalAssets()` until it is explicitly swept into the vault via `getUnstakedFunds` during a rebalance cycle.



```
function totalStaked() external view override returns (uint256 stakedTotal) {
    return _aggregateState.stakedAmount + _aggregateState.pendingUnstakeAmount +
    _pendingClaimAmount;
}
```

That sweep only runs if `rebalance` proceeds past its no-op gate. In a quiet state, `_hasRebalanceWorkAvailable` returns `false` because it never inspects `stakingAsset.balanceOf(StakingManager)`:

```
function _hasRebalanceWorkAvailable() internal view returns (bool) {
    uint256 rewardsAccumulatorBalance = _getRewardsAccumulatorBalance();
    if (rewardsAccumulatorBalance > 0) return true;

    if (_modules.stakingManager.getClaimableRewards() > 0) return true;

    if (_modules.stakingManager.hasFinalizedUnstakes()) return true;
    ...

    return false;
}
```

The refund therefore remains stranded on `StakingManager`, excluded from accounting. `totalAssets()` understates real holdings, reducing the exchange rate for anyone depositing or redeeming while the funds are idle.

It is recommended to extend `_hasRebalanceWorkAvailable` to flag an unswept `StakingManager` balance (`stakingAsset.balanceOf(address(_modules.stakingManager))`) as pending work.

[L-02] `refreshAttesterState` can finalize exit before GSE withdrawal unlocks

Description

`StakingManager._refreshSingleAttester` treats an exit as claimable based on `view_.exit.exitableAt` (derived from the rollup's local `exitDelay`) and immediately calls rollup. `finalizeWithdraw(attester)`:

```
function _isExitExitable(AttesterView memory view_) internal view returns (bool) {
    if (!view_.exit.exists) {
        return false;
    }
    // Timestamp used only to gate exit readiness from the rollup state.
    // slither-disable-next-line timestamp
    >>> return Timestamp.unwrap(view_.exit.exitableAt) < block.timestamp + 1;
}
```

```
function _refreshSingleAttester(IAztecRollup canonicalRollup, address attester) internal
{
    ...
    } else if (_isExitExitable(view_)) {
        // Exitable -- finalize the exit
    }
```



```

        uint256 exitAmount = view_.exit.amount;
        uint256 pendingExit = info.pendingExitAmount;

        _removeAttester(attester);
        emit UnstakeFinalized(attester, exitAmount);
        // slither-disable-next-line calls-loop
        rollup.finalizeWithdraw(attester);
    }
    ...
}

```

However, finalization is routed through `Governance`, which enforces its own unlock delay computed as `votingDelay / 5 + votingDuration + executionDelay`. If this delay exceeds the rollup's `exitDelay`, the governance level check reverts:

`ValidatorOperationsExtLib.finalizeWithdraw` :

```

function finalizeWithdraw(address _attester) internal {
    ...

    delete store.exits[_attester];

>>> store.gse.finalizeWithdraw(exit.withdrawalId);
    store.stakingAsset.safeTransfer(exit.recipientOrWithdrawer, exit.amount);

    emit IStakingCore.WithdrawFinalized(_attester, exit.recipientOrWithdrawer, exit.amount);
}

```

In the `Governance` contract of GSE:

```

function getWithdrawalDelay(CompressedConfiguration storage _self) internal view returns
(Timestamp) {
    Timestamp votingDelay = _self.votingDelay.decompress();
    Timestamp votingDuration = _self.votingDuration.decompress();
    Timestamp executionDelay = _self.executionDelay.decompress();

>>> return Timestamp.wrap(Timestamp.unwrap(votingDelay) / 5) + votingDuration +
    executionDelay;
}

```

```

function initiateWithdraw(address _to, uint256 _amount) external override(IGovernance)
returns (uint256) {
>>> return _initiateWithdraw(msg.sender, _to, _amount, configuration.getWithdrawalDelay());
}

```

```

function _initiateWithdraw(address _from, address _to, uint256 _amount, Timestamp _delay)
internal returns (uint256) {
    ...

    withdrawals[withdrawalId] = Withdrawal({
>>> amount: _amount, unlocksAt: Timestamp.wrap(block.timestamp) + _delay, recipient: _to,
    claimed: false
    });

    ...
}

```



```
function finalizeWithdraw(uint256 _withdrawalId) external override(IGovernance) {
    ...
    require(
>>> Timestamp.wrap(block.timestamp) >= withdrawal.unlocksAt,
        Errors.Governance__WithdrawalNotUnlockedYet(Timestamp.wrap(block.timestamp),
withdrawal.unlocksAt)
    );
    ...
}
```

Mainnet parameters illustrate the mismatch:

- Rollup exit delay: `345,600s` = 4 days
- Governance withdrawal delay: `259,200 / 5 + 604,800 + 2,592,000 = 3,248,640s` ≈ 37 days 14h
- Gap: ~33 days 14h

This creates a mismatch of 2,903,040 seconds (~33 days). So after day 4, Olla may already consider the exit exitable and start calling `finalizeWithdraw()`, but governance may still reject finalization for more than another 33 days. During that period, `refreshAttesterState()` can keep failing on that attester, delaying liquidity realization into the vault.

It is recommended to gate finalization on the actual governance withdrawal unlock for `exit.withdrawalId`.

Reference:

Canonical Rollup: [https://etherscan.io/address/](https://etherscan.io/address/0xae2001f7e21d5ecabf6234e9fdd1e76f50f74962#readContract)

0xae2001f7e21d5ecabf6234e9fdd1e76f50f74962#readContract

Governance: [https://etherscan.io/address/](https://etherscan.io/address/0x1102471Eb3378FEE427121c9EfcEa452E4B6B75e#readContract)

0x1102471Eb3378FEE427121c9EfcEa452E4B6B75e#readContract

[L-03] Instant redemption lacks slippage protection

Description

Similar to the `mint()` / `deposit()` parity gap, `instantRedeem()` (which calls `_redeem()`) has no slippage parameter:

```
function _redeem(address owner, uint256 shares, address recipient, bool
sharesPulledToVault)
    internal returns (uint256 netAssets)
{
    ...
    uint256 rate = coreRef.exchangeRate();
    uint256 grossAssets = coreRef.convertToAssets(shares);
    uint256 fee = grossAssets * instantRedemptionFeeBP / BP_DIVISOR;
    netAssets = grossAssets - fee;
    ...
}
```



If the exchange rate drops between transaction submission and execution (for example, because of a `rebalance()` that recognizes slashing), the caller receives fewer assets than expected with no ability to revert. The async `requestRedeem()` path stores the rate at request time, providing implicit slippage protection, but the instant path does not. Add a `minAssetsOut` parameter to `instantRedeem()` and revert if `netAssets < minAssetsOut`.

[L-04] Defensive underflow clamping masks aggregate state desynchronization

Description

These appear in `purgeFailedQueueEntry()` (line 299), `_initiateUnstake()` (line 532), `_refreshSingleAttester()` (line 598), and `_refreshSingleAttester()` for `pendingUnstakeAmount` (line 661). While each individual clamp prevents a revert, the silent truncation to zero masks a real invariant violation: the aggregate should always equal the sum of per-attester values. When the clamp fires, the aggregate permanently understates the true total, which propagates into `totalStaked()` and then into `OllaCore._computeTotalAssets()`, causing `totalAssets()` to be understated. Consider emitting an event when the else branch executes so that off-chain monitoring can detect the desynchronization and trigger corrective action.

[L-05] Mint function lacks slippage protection

Description

The `deposit()` function includes a `minSharesOut` slippage parameter:

```
function deposit(uint256 assets, address recipient, uint256 minSharesOut) external ... {
    shares = _deposit(msg.sender, assets, recipient);
    if (shares < minSharesOut) revert OllaVault__SlippageExceeded(shares, minSharesOut);
}
```

However, the `mint()` function has no equivalent `maxAssetsIn` parameter:

```
function mint(uint256 shares, address receiver) external ... returns (uint256 assets) {
    ...
    assets = coreRef.convertToAssetsCeil(shares);
    ...
    _processDeposit(msg.sender, assets, shares, receiver);
}
```

If the exchange rate changes between transaction submission and execution (for example, because of a `rebalance()` or `updateAccounting()` call in the same block), the caller may pay more assets than expected for the requested shares. Add a `maxAssetsIn` parameter to `mint()` and revert if `assets > maxAssetsIn`.



[L-06] `renounceOwnership()` is not disabled allowing permanent loss of admin control

Description

`OllaCore` and `OllaVault` inherit `Ownable2StepUpgradeable`, and renouncing ownership via `renounceOwnership()` is possible since the function is not overridden. Leaving the contract without an owner would permanently disable all administrative functions, including setting protocol fees, updating the safety module, configuring the buffered asset target, reconciling assets, recovering tokens, and authorizing contract upgrades.

Consider overriding `renounceOwnership()` to revert when called by placing the following code in `OllaCore` and `OllaVault`:

```
function renounceOwnership() public override onlyOwner {
    revert("renouncing ownership not allowed");
}
```

[L-07] `OllaVault.recoverStAztec` gated by `whenNotPaused` blocks emergency recovery

Description

`OllaVault.recoverStAztec` is the only emergency recovery path for stranded stAztec, and it is gated on `whenNotPaused`. During a genuine incident where the vault is paused precisely because something is broken, the recovery path is unavailable until governance unpauses. There is genuinely no need for this admin recovery function to be pauseable.

Remove the `whenNotPaused` modifier so recovery remains possible during an incident.

[L-08] `_refreshSingleAttester` does not return after `_removeAttester` call execution

Description

In the `Exiting && !view_.exit.exists` branch at `StakingManager.sol:L643-670`, `_removeAttester(attester)` is called but execution falls through to `info.stakedAmount = newBalance;` at L702 and also `emit AttesterStateRefreshed`. The two other removal branches (Active -> zero-balance at L634-638 and exitable finalization at L671-697) both `return` explicitly after `_removeAttester`.

Currently this does not create an issue, because `_processUnstakeAttester` zeros `info.stakedAmount` at exit initiation and Aztec returns `effectiveBalance == 0`; however, the contract is upgradeable and in future versions this might be missed. `info.stakedAmount` might be updated and it is going to be an issue.



Add `return` immediately after `_removeAttester(attester)` in the externally finalized branch (plus an observability event).

[L-09] StakeSurplus leaves nonzero allowance on StakingManager after partial stake

Description

`OllaCore._stakeSurplus` sets `assetRef.forceApprove(address(stakingManagerRef), stakeable)` before calling `stakingManagerRef.stake(stakeable)`. On the normal partial-stake path, excess tokens are returned via `safeTransfer`, but the allowance `stakeable - actualStaked` persists on `asset` for `stakingManagerRef`. In the `catch` branch, the excess tokens are returned, but the entire `stakeable` allowance remains live.

Reset the allowance to zero via `forceApprove(stakingManagerRef, 0)` on both the partial-stake continuation and the `catch` branch before returning from `_stakeSurplus`.

[L-10] Non-upgradeable `ReentrancyGuard` used in upgradeable contract

Description

Several upgradeable contracts inherit from OpenZeppelin's non-upgradeable `ReentrancyGuard` instead of `ReentrancyGuardUpgradeable`:

- `contracts/src/core/OllaCore.sol:13,37`
- `contracts/src/core/RewardsAccumulator.sol:10,20`
- `contracts/src/vault/OllaVault.sol:14,34`
- `contracts/src/vault/WithdrawalQueue.sol:9,26`
- `contracts/src/staking/StakingManager.sol:11,25`
- `contracts/src/staking/StakingProviderRegistry.sol:8,21`

All of these contracts are upgradeable (they inherit `Initializable` / `UUPSUpgradeable` and use the upgradeable variants of the other OpenZeppelin modules).

Recommendation: Replace `ReentrancyGuard` with OpenZeppelin's `ReentrancyGuardUpgradeable` in all of the contracts listed above, import it from `@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol`, and call `__ReentrancyGuard_init()` from each contract's initializer.



[L-11] Fee-rate changes apply retroactively to accrued rewards

Description

The `Olla::core::calculateProtocolFees()` computes the fee as `grossAssetRewards * protocolFeeBP / BP_DIVISOR` using the current storage value of `protocolFeeBP`:

```
// OllaCore.sol:1220
ollaProtocolFeeAssets = grossAssetRewards * protocolFeeBP / BP_DIVISOR;
```

The `grossAssetRewards` represents rewards that accrued over the entire period between the previous report and the current `updateAccounting()` call (derived in `_computeGrossRewards()` from the delta between `oldTotalAssets` and `newTotalAssets`, minus `exitFeesThis Period`).

The issue is that `OllaCore::setProtocolFeeBP()` only writes the new rate to storage; it does not call `_updateAccountingInternal()` to settle rewards at the pre-change rate:

```
// OllaCore.sol:228-233
function setProtocolFeeBP(uint256 newFeeBP) external override onlyOwner whenNotPaused
whenRebalanceDone {
    if (newFeeBP > MAX_PROTOCOL_FEE_BP) revert OllaCore__InvalidFeeBP(newFeeBP);
    uint256 oldFeeBP = protocolFeeBP;
    protocolFeeBP = SafeCast.toUint16(newFeeBP);
    emit ProtocolFeeUpdated(oldFeeBP, newFeeBP);
}
```

As a consequence, on the first `updateAccounting()` after a fee change, rewards that accrued while the old rate was in effect are taxed at the new rate. A legitimate fee increase from, e.g., 10% → 20% converts 10% of the accrued, but unreported reward pool from `stAztec` holders to the treasury/provider recipients, despite those rewards having been earned under the lower fee regime. A similar issue can be observed if the fee split is changed in `setTreasuryFeeSplitBP()`.

It is recommended to update the internal accounting before changing the protocol fee:

```
function setProtocolFeeBP(uint256 newFeeBP) external override onlyOwner whenNotPaused
whenRebalanceDone {
    if (newFeeBP > MAX_PROTOCOL_FEE_BP) revert OllaCore__InvalidFeeBP(newFeeBP);
+   _updateAccountingInternal();
    uint256 oldFeeBP = protocolFeeBP;
    protocolFeeBP = SafeCast.toUint16(newFeeBP);
    emit ProtocolFeeUpdated(oldFeeBP, newFeeBP);
}
```




[L-12] `requestRedeem()` is missing ERC-20 approve authorization path

Description

The ERC-7540 spec for `requestRedeem()` states:

Redeem Request approval of shares for a `msg.sender` NOT equal to owner may come either from ERC-20 approval over the shares of owner or if the owner has approved the `msg.sender` as an operator."

The vault checks only ERC-7540 operator status. It does not check ERC-20 allowance (`stAztec.allowance(owner, msg.sender)`). An integrator contract that obtained ERC-20 approval from the share owner (the standard DeFi composability pattern) would be unable to call `requestRedeem` on their behalf, breaking the expected ERC-7540 authorization model.

It is recommended to extend authorization check to consume `stAztec` allowance:

```
if (msg.sender != owner && !_operators[owner][msg.sender]) {
    // ERC-7540 allows ERC-20 approval on shares; consume allowance via transferFrom
    IERC20(address(_modules.stAztec)).safeTransferFrom(owner, address(this), shares);
    (requestId, assets) = _executeRedeemRequest(owner, controller, controller, shares,
true);
} else {
    (requestId, assets) = _executeRedeemRequest(owner, controller, controller, shares,
false);
}
```

[L-13] Claimed field in `WithdrawalRequest` is dead storage that breaks ERC-7540 spec

Description

The only way `req.claimed` is observable is via `getRequest`. However, no path in the current control flow ever sets this value to `true`. The `WithdrawalQueue::getRequest()` reverts the moment `_requests[id]` is deleted, so the bool never reaches a reader with a value of `true`. Every `!req.claimed` expression in `OllaVault` is therefore trivially true.

```
function pendingRedeemRequest(uint256 requestId, address controller)
    external
    view
    override
    returns (uint256 pendingShares)
{
    if (_requestOwners[requestId] != controller) return 0;
    IWithdrawalQueue.WithdrawalRequest memory request =
    _modules.withdrawalQueue.getRequest(requestId);
    if (!request.finalized && !request.claimed) return request.shares;
    return 0;
}
```



ERC-7540 requires `pendingRedeemRequest()` to return 0 for non-pending / non-claimable ids, not to revert. Off-chain integrators probing id ranges with `controller = address(0)` (common for generic indexers and ERC-7540 scanners) will hit a revert instead:

MUST NOT revert unless due to integer overflow caused by an unreasonably large input.

[L-14] `requestRedeem()` and `requestRedeemWithPermit()` lack `isPaused` safety check

Description

The `OllaVault::requestRedeem()` and `OllaVault::requestRedeemWithPermit()` functions allow users to request an asynchronous redemption, which will be disbursed later after the withdrawal queue is finalized.

In various critical functions such as `_deposit()`, `mint()`, and `_redeem()`, the vault correctly verifies the safety module's `isPaused()` state.

The safety module circuit breaker can be enabled due to the following reasons:

1. Rate Drop: Exchange rate falls by more than `minRateDropBps` threshold (e.g., 5%). Detects validator slashing that devalues all shares.
2. Queue Ratio Breach: Pending withdrawals exceed `maxQueueRatioBps` of gross assets (e.g., 60%). Signals a potential bank run or liquidity crunch.
3. Accounting Staleness: No rebalance update for longer than `maxAccountingDelay` (e.g., 1 day). The exchange rate is stale and cannot be trusted for pricing.

However, a similar check is missing inside `requestRedeem()` and `requestRedeemWithPermit()`, which can be crucial:

```
function _executeRedeemRequest(
    address shareOwner,
    address controller,
    address recipient,
    uint256 shares,
    bool sharesPulledToVault
) internal returns (uint256 requestId, uint256 assetsExpected) {
    if (shares == 0) revert OllaVault__InvalidAmount();

    VaultModules memory modules = _modules;
    IOllaCore coreRef = IOllaCore(modules.core);

    uint256 rate = coreRef.exchangeRate(); <<@
    assetsExpected = coreRef.convertToAssets(shares); <<@
    ISafetyModule(_safetyModule()).checkWithdrawalMinimum(shares); <<@
    uint256 expectedRequestId = modules.withdrawalQueue.nextRequestId();
    // . . .
```



It allows the `maxQueueRatioBps` breach that the pause was meant for stopping the new queue growth, but `requestRedeem()` keeps adding `assetsExpected` to `totalPendingAssets` / `cumulativeWithdrawals`, perpetuating the condition that triggered the pause.

Recommendations

It is recommended to add the `sm.isPaused()` check inside `_executeRedeemRequest()` as it is used by both `requestRedeem()` and `requestRedeemWithPermit()`.

[L-15] Governance role propagation revokes from wrong address

Description

`_propagateAdminRole` is called with `oldGovernance = governanceAdmin`, in which `governanceAdmin` can be different from the OllaGovernance contract itself. However, the satellites were initialized with `DEFAULT_ADMIN_ROLE` granted to the OllaGovernance contract address, not the `governanceAdmin`. The revoke calls therefore target an address that does not hold the role:

```
function _propagateAdminRole(address oldGovernance, address newGovernance) internal {
    // oldGovernance = governanceAdmin (EOA)
    // But satellites hold DEFAULT_ADMIN_ROLE on address(this) (the contract)
    ...
    _propagateSatelliteAdminRoles(satellites, oldGovernance, newGovernance);
}
```

All six `revokeRole` calls are silent no-operations because `AccessControl.revokeRole` does not revert when the target does not hold the role.

[L-16] Stuck rebalance step 5 on buffer drain forces rebalance revert

Description

During rebalance step 5 (`StakeSurplus`), on the first-entry path (when `stakeRemaining == 0`), the code computes `stakeRemaining` from the current buffer surplus. If `stakeRemaining > 0`, it proceeds to `_stakeSurplus`, which calls `vaultRef.transferToCore(stakeRemaining)`. However, if the vault buffer was drained between the `_computeStakeRemaining` read and the `transferToCore` call (for example by an instant redemption in a prior transaction in the same block), `transferToCore` reverts because the vault does not have a sufficient balance.

```
if (progress.stakeRemaining == 0) {
    (uint256 requiredBuffer,) = _computeRequiredBuffer();
    progress.stakeRemaining = _computeStakeRemaining(requiredBuffer);
    if (progress.stakeRemaining == 0) {
```



```
        progress.step = IOllaCore.RebalanceStep.Done;
    }
}
```

Recovery is possible via `forceRebalanceReset()`, but the rebalance is stuck until the guardian intervenes. Add a `canStake` check on the first-entry path to match the continuation path at L424.

[L-17] Fully slashed attester DoSes `rebalance` until permissionless refresh clears it

Description

`StakingManager._processUnstakeAttester` calls `rollup.initiateWithdraw(attester, address(this))` and branches on the returned `bool`, expecting a graceful skip when the rollup refuses. The real Aztec `StakingLib.initiateWithdraw` has exactly one `return true` (at `StakingLib.sol:L464`); every failure path is a `require` that reverts. The reachable revert for an Olla operated attester is `!exit.exists && effectiveBalance == 0`, which `StakingLib.slash` produces when the full balance is slashed (`toUser = amountWithdrawn - slashAmount == 0` skips the exit-record creation and the attester is removed from the GSE with `effectiveBalance == 0`).

Until a permissionless `refreshAttesterState([slashedAttester])` cleans the zombie out of Olla's `_activeAttesterSet`, `StakingManager.unstake` will iterate the active set, hit the slashed attester, and revert. No `try/catch` exists on the call path, so the revert propagates through `_initiateUnstake` and reverts the entire `rebalance()`. Harvest, PullUnstaked, and FinalizeWithdrawals done in the same transaction all roll back. User withdrawal-queue entries cannot finalize, sequencer rewards are not harvested, and no new unstakes can be initiated until someone refreshes the attester.

Recommendation

Wrap the external call in `try/catch` so the graceful skip intent fires against the real Aztec contract:

```
- bool isInitiated = rollup.initiateWithdraw(attester, address(this));
- if (!isInitiated) {
-     if (view_.exit.exists) {
-         _setAttesterStatus(attester, info, InternalAttesterStatus.Exiting);
-         return 0;
-     }
-     revert StakingManager__UnstakeFailed(attester);
- }
+ try rollup.initiateWithdraw(attester, address(this)) returns (bool) {
+     // continue to the main path
+ } catch {
+     if (view_.exit.exists || view_.effectiveBalance == 0) {
+         _setAttesterStatus(attester, info, InternalAttesterStatus.Exiting);
+         if (view_.effectiveBalance == 0) _removeAttester(attester);
+         return 0;
+     }
+ }
```



```
+     }  
+     revert StakingManager__UnstakeFailed(attester);  
+ }
```

[L-18] Vault ownership drift permanently bricks WithdrawalQueue UUPS upgrades

Description

`WithdrawalQueue._authorizeUpgrade` carries a dual check: the caller must hold `DEFAULT_ADMIN_ROLE` and `msg.sender` must equal `OwnableUpgradeable(vault).owner()`. It was added as defense-in-depth so a lone compromise of `DEFAULT_ADMIN_ROLE` cannot upgrade the queue. In isolation that is a reasonable hardening choice.

The problem is a hidden cross-contract coupling.

`OllaGovernance.upgradeSatellite(proxy, newImpl)` delegates the authorization check to the target. For every other satellite the check is `DEFAULT_ADMIN_ROLE` only, which governance holds permanently. For `WithdrawalQueue` it additionally requires `vault.owner() == governance`. Today that is true, but `OllaVault` inherits `Ownable2StepUpgradeable` and `transferOwnership` is a standard `onlyOwner` function, so governance can schedule a transfer through the timelock. Once the new owner calls `acceptOwnership`, `vault.owner()` no longer returns the governance proxy and `upgradeSatellite(withdrawalQueue, newImpl)` reverts until ownership is returned.

Nothing in the code asserts or documents this coupling, so a legitimate-looking governance action (a migration, a multisig reshuffle, a restructuring) silently locks queue upgrades.

Recommendation

Compare `_authorizeUpgrade` against a stored governance address rather than reading through `vault.owner()`. Set it at initialization and either make it immutable or governance-rotatable via `onlySelf`. The intent is "governance can upgrade the queue", which should hold regardless of who owns the vault.

[L-19] `setOperator` is gated by `whenNotPaused` which can create weakness

Description

`OllaVault.setOperator` uses the `whenNotPaused` modifier, which prevents any controller from changing an operator approval while the vault is paused:

```
// contracts/src/vault/OllaVault.sol:342-347  
function setOperator(address operator, bool approved) external override whenNotPaused  
returns (bool) {  
    if (operator == msg.sender) revert OllaVault__InvalidParameter();  
}
```



```
_operators[msg.sender][operator] = approved;  
emit OperatorSet(msg.sender, operator, approved);  
return true;  
}
```

If a controller suspects one of their approved operators has been compromised, or simply wants to rewind an approval before normal activity resumes, under the current gating that controller cannot act for the entire duration of the pause. The moment the vault is unpaused, the still-approved operator can immediately call `instantRedeem` / `requestRedeem` on behalf of the controller before the controller has any chance to revoke them, which defeats a core defensive purpose of the pause.

Revocation primitives should generally remain callable under pause because blocking them is strictly safety-reducing. Approvals being granted while paused is the only direction with real risk, and that can be handled with narrower logic than `whenNotPaused`.

Recommendation: Remove the `whenNotPaused` modifier from `setOperator` so controllers can revoke operators during the paused state. If approvals (`approved = true`) should still be blocked while paused, gate only that branch explicitly and always allow `approved = false`.

[L-20] `setSafetyModule` does not update `lastAccountingTimestamp` on new safety module

Description

The `OllaCore::setSafetyModule()` allows the owner to update the safety module:

```
/// @inheritdoc IOllaCore  
function setSafetyModule(address newSafetyModule) external override onlyOwner  
whenNotPaused whenRebalanceDone {  
    address oldSafetyModule = GovernanceLib.setSafetyModule(_modules, newSafetyModule);  
    emit SafetyModuleUpdated(oldSafetyModule, newSafetyModule);  
}
```

However, this call fails to update the `lastAccountingTimestamp`, which can allow for the next `checkAccountingLiveness` to either trip immediately (false DoS) or grant a free stale window.

It is recommended to update the timestamp within the setter call via the `_updateAccountingInternal()` function call.



[L-21] WithdrawalMinimum is share denominated

Description

Currently, the `withdrawalMinimum` is share-denominated. When slashing occurs, the exchange rate drops, and a user's same share count now represents fewer assets. A fixed share-denominated minimum means the asset floor shrinks proportionally. Small users can end up below what the protocol's authors presumably intended as the minimum efficient withdrawal.

It is recommended to denominate `withdrawalMinimum` in assets and convert to shares at each check, or track both in parallel.

[L-22] `getClaimableRewards()` returns full value when `isRewardsClaimable` is false

Description

The `OllaCore::harvestRewards()` function harvests the accumulated rewards if the rollup's `isRewardsClaimable` flag is set to `true`; otherwise, it returns `0`:

```
function harvestRewards() external override onlyCore nonReentrant returns (uint256
harvested) {
    (, IAztecRollup rollup) = _getRollup();
    try rollup.claimSequencerRewards(rewardsAccumulator) returns (uint256 claimed) {
        harvested = claimed;
    } catch (bytes memory reason) {
        emit RewardsHarvestFailed(reason);
        return 0;
    }
    emit RewardsHarvested(harvested);
    return harvested;
}
```

```
// contracts/dependencies/aztec-contracts-4.0.0/src/core/libraries/rollup/RewardLib.sol
function claimSequencerRewards(address _sequencer) internal returns (uint256) {
    RewardStorage storage rewardStorage = getStorage();
    require(rewardStorage.isRewardsClaimable, Errors.Rollup__RewardsNotClaimable()); <<@
```

In a case where this flag is set to `false`, the `StakingManager::getClaimableRewards()` view function still returns the accumulated amount even though it is currently not claimable:

```
function getClaimableRewards() external view override onlyCore returns (uint256
claimableRewards) {
    (, IAztecRollup rollup) = _getRollup();
    return rollup.getSequencerRewards(address(rewardsAccumulator)); // <- returns full
accrued even when flag=false
}
```



This inconsistency can misguide any off-chain entity using the view call and temporarily inflate `totalAssets()`, which will be in sync with the intentions as soon as the flag is switched back.

It is recommended to return `0` if the `isRewardsClaimable` flag is set to `false`:

```
function getClaimableRewards() external view override onlyCore returns (uint256) {
    (, IAztecRollup rollup) =
    _getRollup();

    +   if (!rollup.isRewardsClaimable()) return 0;
        return
    rollup.getSequencerRewards(address(rewardsAccumulator));

}
```

[L-23] Rebalance buffer math ignores `_pendingClaimAmount`

Description

The `StakingManager` tracks unstaked capital in two fields: `_aggregateState.pendingUnstakeAmount`, which are funds requested from the rollup, not yet finalized, and `_pendingClaimAmount`, which are the funds the rollup has finalized but that `OllaCore` has not yet pulled into the vault.

The rebalance state machine flows:

```
Harvest → PullUnstaked → FinalizeWithdrawals → InitiateUnstake → StakeSurplus
```

The `PullUnstaked` is the only step that drains `_pendingClaimAmount` into the vault buffer.

1. Cycle `N` calls `rebalance()` and completes `PullUnstaked` (buffer receives whatever was in `_pendingClaimAmount` at that instant).
2. Cycle `N` is gas-interrupted at the `FinalizeWithdrawals` step, or if `_rebalanceProgress` saves the step and returns:

```
// OllaCore.sol:362-375
if (progress.step == IOllaCore.RebalanceStep.FinalizeWithdrawals) {
    if (!_hasGasForStep()) {
        _rebalanceProgress = progress; // <-- saves step, returns early
        return (rewardsDelta, 0, 0, vaultRef.bufferedAssets());
    }
    ...
}
```

1. Between cycle `N`'s interrupt and cycle `N+1`, the rollup finalizes one or more exits. `StakingManager` moves the amount `pendingUnstakeAmount` → `_pendingClaimAmount`:



```
// StakingManager.sol:661-666 (externally-finalized path)
if (_aggregateState.pendingUnstakeAmount >= pendingExit) {
    _aggregateState.pendingUnstakeAmount -= pendingExit; // <-- pendingUnstakeAmount drops
} else {
    _aggregateState.pendingUnstakeAmount = 0;
}
_pendingClaimAmount += pendingExit; // <-- _pendingClaimAmount rises
```

1. Cycle **N+1** resumes at the saved step — **PullUnstaked** is not re-entered. On reaching **InitiateUnstake**, the resume path unconditionally recomputes **progress.unstakeRemaining = _computeUnstakeRemaining(requiredBuffer)** :

```
// OllaCore.sol:379-392
if (progress.step == IOllaCore.RebalanceStep.InitiateUnstake) {
    {
        (uint256 requiredBuffer,) = _computeRequiredBuffer();
        progress.unstakeRemaining = _computeUnstakeRemaining(requiredBuffer); // <--
recomputed every resume
    }
    ...
    uint256 initiated = _initiateUnstake(progress.unstakeRemaining); // <--
committed on-chain
}
```

1. The recomputation sees:
2. **currentBuffer** : unchanged (finalized funds are still in **StakingManager**, not vault).
3. **pendingUnstakes** : decreased by the finalized amount.
4. **amountToUnstake - pendingUnstakes** : inflated by exactly that amount.

```
// OllaCore.sol:1133-1145
function _computeUnstakeRemaining(uint256 requiredBuffer) internal view returns (uint256
remaining) {
    uint256 currentBuffer = IOllaVault(_modules.vault).bufferedAssets(); // unchanged -
funds still in StakingManager
    if (requiredBuffer < currentBuffer) return 0;
    uint256 amountToUnstake = requiredBuffer - currentBuffer;
    uint256 pendingUnstakes =
_modules.stakingManager.pendingUnstakes(); // <-- decreased by finalized amount
    if (pendingUnstakes > amountToUnstake) return 0;
    remaining = amountToUnstake -
pendingUnstakes; // <-- inflated by same amount
}
```

1. An unneeded additional unstake is initiated. On cycle **N+2**, **PullUnstaked** pulls in the already finalized amount plus anything newly finalized, overshooting the buffer target. Then the **StakeSurplus** stakes the overshoot back out:

```
// OllaCore.sol:1150-1157
function _computeStakeRemaining(uint256 requiredBuffer) internal view returns (uint256
remaining) {
    uint256 currentBuffer = IOllaVault(_modules.vault).bufferedAssets();
    if (currentBuffer < requiredBuffer) return 0;
```



```

    remaining = currentBuffer - requiredBuffer; // <-- stakes
    the overshoot back out
}

```

Each over-initiated unstake commits attesters to an exit they did not need to make, exposing their stake to slashing during the `EXIT_DELAY` window for no protocol benefit, and forcing additional rotation.

It is recommended to expose `_pendingClaimAmount` from `StakingManager` and use it as a pending incoming funds accounting inside `_computeUnstakeRemaining()` and `_computeStakeRemaining()` to offset the issue.

[L-24] `upgradeCore()` and `upgradeSatellite()` hardcode empty init data

Description

Both upgrade functions pass `""` to `upgradeToAndCall`. Any implementation requiring `reinitializeV2(...)` must be initialized in a separate transaction. Between the upgrade and the reinit call, anyone can front-run `reinitializer(n)`

```

function upgradeCore(address newImplementation) external override onlySelf {
    if (newImplementation == address(0)) {
        revert OllaGovernance__ZeroAddress("newImplementation");
    }
    UUPSUpgradeable(core).upgradeToAndCall(newImplementation, ""); <<@
}

/// @inheritdoc IOllaGovernance
function upgradeSatellite(address proxy, address newImplementation) external override
onlySelf {
    if (proxy == address(0)) {
        revert OllaGovernance__ZeroAddress("proxy");
    }
    if (newImplementation == address(0)) {
        revert OllaGovernance__ZeroAddress("newImplementation");
    }
    UUPSUpgradeable(proxy).upgradeToAndCall(newImplementation, ""); <<@
}

```

It is recommended to accept bytes calldata data and forward it or document this behavior if it is intended.

[L-25] Stale exchange rate due to infrequent accounting update

Description

The stAztec/AZTEC exchange rate depends on the timely execution of `refreshAttesterState` followed by `updateAccounting`. According to the documentation, the Butler bot calls these on a fixed schedule:



The table presents information regarding the operations related to the stAztec/AZTEC exchange rate, specifically focusing on their frequency and the conditions under which they are executed.

There are two key operations outlined in the table:

The first operation is `refreshAttesterState`, which is executed every 60 seconds. This operation is triggered when stale or unhealthy attesters are detected, ensuring that the system remains responsive to any issues that may arise with the attesters.

The second operation is `updateAccounting`, which occurs every 5 minutes. This operation is conditioned on the last report being older than the staleness threshold, indicating that it is crucial for maintaining accurate accounting and reflecting the most current state of the system.

Together, these operations are essential for maintaining the integrity and accuracy of the exchange rate, as they ensure timely updates and responsiveness to potential issues.

<http://docs.olla.finance/operations/permissionless-operations#the-butler>

This schedule creates a window during which the on-chain rate does not reflect the actual rollup state, for example, after a slashing event or an attester balance change. Since both functions are permissionless and the exchange rate directly governs deposit and redemption conversions, a sophisticated actor could monitor the rollup for such state changes and front-run the accounting update to deposit or redeem at a favorable stale rate before it corrects.

It is recommended that in addition to the scheduled polling, you monitor rollup events (e.g., slashing, attester exits, balance changes) and trigger `refreshAttesterState` and `updateAccounting` immediately upon detection, rather than waiting for the next scheduled interval.

[L-26] FinalizeWithdrawals fails to increment finalizedCount for zero value requests

Description

In `finalizeWithdrawals`, when `assetsExpected == 0`, the request is correctly marked as `finalized` and `currentId` is incremented, but `finalizedCount` is never incremented before the continue statement.

```
function finalizeWithdrawals(uint256 available, uint256 currentRate)
    external
    override
    onlyVault
    nonReentrant
    returns (uint256 used, uint256 finalizedCount, uint256 totalAdjusted)
{
    ...
    if (assetsExpected == 0) {
        pendingShares_ -= request.shares;
```



```
        request.finalized = true;
        emit WithdrawalFinalized(currentId, 0);
        ++currentId;
        continue;
    }

    ...
}
```

It is recommended to increase `finalizedCount` in the zero-value branch, consistent with the non-zero path.

[L-27] Queued attesters can be incorrectly purged after a rollup upgrade

Description

`purgeFailedQueueEntry` is intended to clean up attesters that were queued for staking but failed to activate on the rollup. However, after a rollup upgrade, this function can be incorrectly triggered for any attester that was queued on the old rollup but has not yet been flushed.

The function fetches the attester's view from the `new` canonical rollup. Because the attester has never been registered on the new rollup instance, it returns `Status.NONE` and `effectiveBalance = 0`. The entry queue check also passes, since the new rollup's entry queue does not store old version records. All three conditions are therefore satisfied, allowing anyone to purge a legitimately queued attester as if it were a failed entry.

```
function purgeFailedQueueEntry(address attester) external override nonReentrant {
    ...
>>>    (, IAztecRollup rollup) = _getRollup(); // fetches NEW canonical rollup
    AttesterView memory view_ = rollup.getAttesterView(attester);
    if (view_.status != Status.NONE || view_.exit.exists || view_.effectiveBalance > 0) {
        revert StakingManager__NotFailedQueueEntry(attester);
    }

    uint256 queueLen = rollup.getEntryQueueLength();
    for (uint256 i; i < queueLen; ++i) {
        // slither-disable-next-line calls-loop
        if (rollup.getEntryQueueAt(i).attester == attester) {
            revert StakingManager__NotFailedQueueEntry(attester);
        }
    }

    ...
}
```

If anyone attempts to purge a legitimately queued attester, the protocol may subtract a queued attester's cached stake before the old rollup queue has actually failed and refunded the stake. This can temporarily understate total staked assets, depress the exchange rate, and



cause queued withdrawals to be adjusted or finalized using a distorted rate. If the old rollup later refunds the funds, they may re-enter as excess / donation-like assets rather than through the intended accounting path.

Recommendations

When an attester is queued via `deposit`, record the rollup address at the time of queuing for verification.

[L-28] Admin role not renounced after initialization

Description

The `initialize` function passes an admin address to `__TimelockController_init`, granting it the timelock admin role. However, this role is never renounced upon completion of initialization.

```
function initialize(
    uint256 minDelay,
    address[] calldata proposers,
    address[] calldata executors,
    address admin,
    address treasury_
) external override initializer {
    if (admin == address(0)) {
        revert OllaGovernance__ZeroAddress("admin");
    }
    if (treasury_ == address(0)) {
        revert OllaGovernance__ZeroAddress("treasury");
    }

>>>    __TimelockController_init(minDelay, proposers, executors, admin);

    treasury = treasury_;
    // The first proposer is the initial governance admin
    if (proposers.length > 0) {
        governanceAdmin = proposers[0];
    }
}
```

As noted in the `__TimelockController_init` NatSpec, the admin role is intended only to aid with initial configuration after deployment and should be subsequently renounced in favor of administration through timelocked proposals. Currently, `DEFAULT_ADMIN_ROLE` remains externally held after setup, allowing direct role administration outside the timelock.

```
/**
 * @dev Initializes the contract with the following parameters:
 *
 * - `minDelay`: initial minimum delay in seconds for operations
 * - `proposers`: accounts to be granted proposer and canceller roles
 * - `executors`: accounts to be granted executor role
```



```

* - `admin`: optional account to be granted admin role; disable with zero address
*
* IMPORTANT: The optional admin can aid with initial configuration of roles after
deployment
* without being subject to delay, but this role should be subsequently renounced in
favor of
* administration through timelocked proposals. Previous versions of this contract would
assign
* this admin to the deployer automatically and should be renounced as well.
*/
function __TimelockController_init(uint256 minDelay, address[] memory proposers,
address[] memory executors, address admin) internal onlyInitializing {
    __TimelockController_init_unchained(minDelay, proposers, executors, admin);
}

```

It is recommended to renounce the admin role at the end of `initialize`:

```

function initialize(...) external override initializer {
    // ...
    __TimelockController_init(minDelay, proposers, executors, admin);
    treasury = treasury_;
    if (proposers.length > 0) {
        governanceAdmin = proposers[0];
    }
+   _revokeRole(DEFAULT_ADMIN_ROLE, admin);
}

```

[L-29] Fee asset and staking asset mismatch can corrupt reward accounting

Description

OllaCore implicitly assumes that `rewardsToken` is the same token as the staking asset. However, `RollupCore` accepts distinct `_feeAsset` and `_stakingAsset` in its constructor, meaning a future rollup upgrade could set them to different tokens:

```

constructor(
>>> IERC20 _feeAsset,
>>> IERC20 _stakingAsset,
    GSE _gse,
    IVerifier _epochProofVerifier,
    address _governance,
    GenesisState memory _genesisState,
    RollupConfigInput memory _config
) Ownable(_governance) {

```

It can lead to corrupting the reward calculation.

It is recommended to add an invariant check at harvest time to assert that the rollup's fee asset matches the staking asset.



[L-30] Has hasRebalanceWorkAvailable does not check for buffer deficit

Description

`_hasRebalanceWorkAvailable` only checks for a buffer surplus (`currentBuffer > targetBufferedAssets`) and never for a buffer deficit. This means that if `currentBuffer < targetBufferedAssets`, the function returns `false` and the cycle is marked idle even when replenishment is needed.

```
function _hasRebalanceWorkAvailable() internal view returns (bool) {
    ...
    uint256 currentBuffer = vaultRef.bufferedAssets();
>>>    if (currentBuffer > targetBufferedAssets) { // deficit case never handled
        uint256 surplus = currentBuffer - targetBufferedAssets;
        if (_modules.stakingManager.canStake(surplus)) return true;
    }

    return false;
}
```

This creates a problematic sequence: a cycle is marked idle → `refreshAttesterState` makes attesters available for unstaking → `rebalance` is called but no-ops because `_hasRebalanceWorkAvailable` returns `false` → the buffer is never replenished.

It is recommended to check for the deficit case before returning `false`.

[L-31] Dust donation to `RewardsAccumulator` bypasses cooldown and idle rebalance check

Description

`OllaCore.rebalance` includes an idle fast-path that skips cooldown consumption when no real work is available. However, `_hasRebalanceWorkAvailable` treats any non-zero `RewardsAccumulator.balance` as actionable:

```
function _hasRebalanceWorkAvailable() internal view returns (bool) {
    uint256 rewardsAccumulatorBalance = _getRewardsAccumulatorBalance();
>>>    if (rewardsAccumulatorBalance > 0) return true;

    ...
}
```

An attacker can donate 1 wei of the asset to the accumulator and call `rebalance`. The harvest step records the donation as `rewardsDelta`, bypasses the idle early return, and refreshes `lastRebalanceTimestamp`.

It is recommended to ignore accumulator balances below a configured minimum threshold inside `_hasRebalanceWorkAvailable`.



[L-32] Out-of-band reward claims desync rewards reporting

Description

`OllaCore._getStakingManagerState` computes `currentRewards` as `_accountingState.cumulativeRewards + claimableRewards`, omitting any balance already sitting in `RewardsAccumulator` that has not yet been recorded via `recordBalance`:

```
function _getStakingManagerState()
  internal
  view
  returns (
    uint256 currentRewards,
    uint256 rewardsDelta,
    uint256 slashingDelta,
    uint256 stakedPrincipal,
    uint256 claimableRewards
  )
{
  IOllaCore.CoreModules memory modules = _modules;
  IOllaCore.AccountingState memory accountingSnapshot = _accountingState;
  claimableRewards = modules.stakingManager.getClaimableRewards();
  >>> currentRewards = accountingSnapshot.cumulativeRewards + claimableRewards;

  uint256 latestReportRewards = _latestReport.rewardsSnapshot;

  // Signed arithmetic for reward delta computation; not a timestamp concern.
  // slither-disable-next-line timestamp
  >>> int256 rewardsDeltaSigned = SafeCast.toInt256(currentRewards) -
  SafeCast.toInt256(latestReportRewards);

  ...
}
```

Because `rollup.claimSequencerRewards(rewardsAccumulator)` is publicly callable, an external actor can transfer Olla's pending rewards into `RewardsAccumulator` before the normal harvest flow. If `updateAccounting` is then called, `claimableRewards` drops while the unrecorded accumulator balance remains invisible to `currentRewards`, driving `rewardsDeltaSigned` negative. This causes `_latestReport.rewardsSnapshot` to be written with an understated value.

When Core subsequently harvests, `recordBalance` treats the pre-existing accumulator balance as fresh rewards, producing an inflated `rewardsDelta` that is inconsistent with the prior snapshot, resulting in incorrect reward reporting through accounting updates.

It is recommended to include the unrecorded `RewardsAccumulator` balance in `currentRewards`:

```
currentRewards =
  accountingSnapshot.cumulativeRewards + claimableRewards +
  _getRewardsAccumulatorBalance();
```




[L-33] RedeemRequest event emits assets instead of shares violating ERC-7540

Description

ERC-7540 specifies that the `RedeemRequest` event MUST emit the `shares` being requested as its final parameter:

```
event RedeemRequest(address indexed controller, address indexed owner, uint256 indexed requestId, address sender, uint256 shares);
```

<https://eips.ethereum.org/EIPS/eip-7540#RedeemRequest>

However, `OllaVault.requestRedeem()` and `OllaVault.requestRedeemWithPermit()` both pass `assets` (the converted asset amount) as the last argument.

It is recommended to emit the `shares` parameter that was passed into the request.

[L-34] Externally finalized exits can be swept before updating `_pendingClaimAmount`

Description

`getUnstakedFunds` sweeps StakingManager's full balance, but only reports `_pendingClaimAmount` as the exit amount:

```
function getUnstakedFunds() external override onlyCore nonReentrant returns (uint256 received, uint256 exitAmount) {
    exitAmount = _pendingClaimAmount;
    _pendingClaimAmount = 0;

    >>> uint256 balance = stakingAsset.balanceOf(address(this));
    if (balance > 0) {
        stakingAsset.safeTransfer(core, balance);
        emit UnstakedFundsClaimed(balance);
    }
    received = balance;
    return (received, exitAmount);
}
```

If the rollup-side exit was finalized (sending assets to StakingManager), then `rebalance` is called to sweep StakingManager's balance to the vault, and `refreshAttesterState()` runs after that. The `Exiting + !view_.exit.exists` adds `_pendingClaimAmount += pendingExitAmount`. At that point, the funds have already been transferred to the vault, but the exit amount is still counted as pending until a later rebalance clears `_pendingClaimAmount`.

```
function _refreshSingleAttester(IAztecRollup canonicalRollup, address attester) internal
{
    ...
}
```



```

        if (info.status == InternalAttesterStatus.Exiting) {
            if (!view_.exit.exists) {
                uint256 pendingExit = info.pendingExitAmount;
                if (_aggregateState.pendingUnstakeAmount >= pendingExit) {
                    _aggregateState.pendingUnstakeAmount -= pendingExit;
                } else {
                    _aggregateState.pendingUnstakeAmount = 0;
                }
            }
            _pendingClaimAmount += pendingExit;
            _removeAttester(attester);
            ...
        }
    }
}

```

It is not that the pending amount overstates the actual finalized amount, but the amount is double counted.

Since `totalAssets` includes both `_pendingClaimAmount` and assets received in the buffer, this can overstate `totalAssets()` and inflate the exchange rate for at least one accounting window, mispricing deposits and instant redemptions.

Recommendations

The `Exiting + !view_.exit.exists` path should not add to `_pendingClaimAmount` if the corresponding funds were already swept. It should instead only clear the stale pending-exit accounting and remove the attester.

[L-35] StakeSurplus step skips canStake check on fresh entry

Description

In `rebalance`, the `canStake` guard is only evaluated when resuming a partial stake (when `progress.stakeRemaining != 0`). On a fresh entry to StakeSurplus where `progress.stakeRemaining == 0`, the code computes a new `stakeRemaining` via `_computeStakeRemaining` and only short-circuits to `Done` if that value is strictly zero.

It never calls `canStake`, so execution falls through to `_stakeSurplus` even when there are no available attester keys or the amount is below the rollup's activation threshold.

```

function rebalance()
    external
    override
    whenNotPaused
    nonReentrant
    returns (uint256 rewardsDelta, uint256 finalizedAmount, uint256 stakedAmount,
uint256 resultingBuffer)
    {
        ...
        if (progress.step == IOllaCore.RebalanceStep.StakeSurplus) {
            if (progress.stakeRemaining == 0) {
                (uint256 requiredBuffer,) = _computeRequiredBuffer();
                progress.stakeRemaining = _computeStakeRemaining(requiredBuffer);
                // slither-disable-next-line incorrect-equality,timestamp
            }
        }
    }
}

```



```

        if (progress.stakeRemaining == 0) {
            progress.step = IOllaCore.RebalanceStep.Done;
        }
        // No canStake() check on fresh entry – falls through to _stakeSurplus
    } else if (!_modules.stakingManager.canStake(progress.stakeRemaining)) {
        ...
    }
    if (progress.step == IOllaCore.RebalanceStep.StakeSurplus) {
        if (!_hasGasForStep()) {
            _rebalanceProgress = progress;
            return (rewardsDelta, finalizedAmount, 0, vaultRef.bufferedAssets());
        }
    }

    stakedAmount = _stakeSurplus(progress.stakeRemaining);
    progress.stakeRemaining -= stakedAmount;

    ...
}

```

`stakeSurplus` pulls stakeable from the vault to core, approves the staking manager, and calls `stake`, which either reverts (caught) or returns 0. Every such rebalance wastes gas on unnecessary token transfers and an approval which can lead to `rebalance` unnecessary stop due to `_hasGasForStep` check later.

It is recommended to apply the `canStake` guard symmetrically on fresh entry of `RebalanceStep.StakeSurplus`.

[L-36] SafetyModule breaker stack cannot detect slashing without `refreshAttesterState`

Description

The protocol's slashing detection pipeline has two stages:

- Stage 1 (Rollup → StakingManager): The permissionless `refreshAttesterState()` reads each attester's `effectiveBalance` from the Aztec rollup and updates `_aggregateState.stakedAmount` and `_aggregateState.slashingDelta` inside `StakingManager`.
- Stage 2 (StakingManager → OllaCore → SafetyModule): During `rebalance()` or `updateAccounting()`, `OllaCore._getStakingManagerState()` reads `totalStaked()` and `getSlashingDelta()` from `StakingManager`, then feeds those values into `_validateSlashingDelta()` and `_validateRateDrop()`, which trigger the SafetyModule circuit breaker if thresholds are exceeded.

The problem is that Stage 2 trusts Stage 1 to have already run, but nothing enforces this. `_getStakingManagerState()` reads whatever `StakingManager` currently reports without verifying that a refresh has occurred:

```

// OllaCore.sol L1079-1080
slashingDelta = modules.stakingManager.getSlashingDelta();
stakedPrincipal = modules.stakingManager.totalStaked();

```



And `totalStaked()` is a pure read of cached state:

```
// StakingManager.sol L357-359
function totalStaked() external view override returns (uint256 stakedTotal) {
    return _aggregateState.stakedAmount + _aggregateState.pendingUnstakeAmount +
        _pendingClaimAmount;
}
```

The SafetyModule's `checkAccountingLiveness` breaker only checks whether `_updateAccountingInternal` has been called recently — it does not check whether `refreshAttesterState` has been called:

```
// SafetyModule.sol L196-210
function checkAccountingLiveness() external override onlyCoreOrVault {
    if (block.timestamp <= lastAccountingTimestamp) {
        return;
    }
    uint256 elapsed = block.timestamp - lastAccountingTimestamp;
    if (elapsed > maxAccountingDelay) {
        _triggerBreaker(ISafetyModule.BreakerReason.AccountingStale);
    }
}
```

The `lastAccountingTimestamp` is updated every time `_updateAccountingInternal` completes, regardless of whether any attester refreshes occurred beforehand.

Trigger sequence:

The following conditions must all hold simultaneously for this issue to cause harm:

1. Condition A — Slashing occurs on the Aztec rollup. One or more active attesters are slashed, reducing their `effectiveBalance` on the rollup. The `StakingManager._aggregateState` still reflects the pre-slash balances because `refreshAttesterState` has not been called for the affected attesters.
2. Condition B — No one calls `refreshAttesterState` for the slashed attesters. Although `refreshAttesterState` is permissionless, there is no on-chain incentive (no reward, no bounty) to call it. If no external actor (keeper, MEV bot, or altruistic caller) invokes it for the specific slashed attesters, Stage 1 remains stale indefinitely.
3. Condition C — `rebalance()` or `updateAccounting()` is called while Stage 1 is stale. This is the normal protocol cadence. The accounting pipeline runs, reads the stale (pre-slash) `totalStaked()` and `slashingDelta = 0`, and computes an inflated `totalAssets` and `exchangeRate`. The `_validateSlashingDelta` check passes because `slashingDelta` is 0. The `checkRateDrop` check passes because the rate has not dropped (it is artificially high).
4. Condition D — `_updateAccountingTimestamp` resets the liveness clock. After the accounting run completes, `setLatestAccountingTimestamp(block.timestamp)` is called, resetting the `checkAccountingLiveness` timer. The SafetyModule now considers accounting "fresh" even though the underlying staking data is stale.
5. Condition E — A user calls `instantRedeem()` at the inflated rate. The exchange rate reflects pre-slash `totalAssets`, so the user receives more underlying assets per share than the protocol's actual backing justifies. The loss is socialized across remaining holders.



Conditions A and B together create the stale window. Condition C propagates the stale data into the exchange rate. Condition D prevents the liveness breaker from detecting the problem. Condition E is the extraction path.

The codebase acknowledges this trade-off explicitly but does not acknowledge the consequent breaker blindness:

```
// OllaCore.sol L1092-1096
/// @dev This function checks only protocol-internal state (e.g. `_pendingClaimAmount`) and
/// does NOT
///     query the rollup for unclaimed exits directly. Rollup state must be synced
///     beforehand via
///     the permissionless `refreshAttesterState()` call. This is intentional: on-demand
///     rollup
///     queries would require iterating all attesters with cross-contract calls, creating
///     unbounded
///     gas costs. Staleness between `refreshAttesterState()` calls is an accepted design
///     trade-off.
```

Recommendations

Track a per-attester refresh timestamp in `_aggregateState` and enforce it in the liveness check.

[L-37] Instant-redemption fee accretes into share price

Description

The instant-redemption fee never leaves the vault, letting depositors front run large redeemers and skim the fee from long-term holders. In `OllaVault._redeem`, all redeemed shares are burned, but only `netAssets` is subtracted from `_bufferedAssets`; the fee stays in the buffer:

```
uint256 fee = grossAssets * instantRedemptionFeeBP / BP_DIVISOR;
netAssets = grossAssets - fee;
...
modules.stAztec.burn(..., shares);
_bufferedAssets -= netAssets; // fee remains in the vault
```

Since `totalAssets()` includes `bufferedAssets`, the fee instantly raises the exchange rate for everyone holding shares in that block. There is no deposit fee, no share hold period, and `requestRedeem` locks the current rate at request time with no fee, and that locked rate is only ever adjusted downward on slashing.



Attack path:

1. Attacker sees a large `instantRedeem()` in the mempool.
2. Attacker front-runs with `deposit()` at the pre-redeem rate.
3. User's `instantRedeem` settles - the fee stays in the buffer, shares burned, share price jumps.
4. Attacker back-runs with `requestRedeem`, locking the post-jump rate at no fee.
5. Attacker claims after queue finalization and profits.

```
function test_PoC_RateJumpsAfterInstantRedeem() external {
    // Long-term holder seeds the vault with a large deposit at the initial 1:1 rate.
    uint256 longTermDeposit = 1_000 * DECIMALS;
    _performDeposit(alice, longTermDeposit);

    uint256 rateBefore = core.exchangeRate();

    emit log_named_uint("rate BEFORE (1e18-scaled)", rateBefore);

    // Large redeemer shows up: redeems 500 shares at 1:1 -> gross=500e18, fee=25e18
    (5%).
    uint256 sharesToRedeem = 500 * DECIMALS;
    uint256 rateSnap = core.exchangeRate();
    uint256 grossAssets = sharesToRedeem.mulDiv(rateSnap, DECIMALS, Math.Rounding.Floor);
    uint256 fee = grossAssets * 500 / BP_DIVISOR;
    uint256 netAssets = grossAssets - fee;

    vm.prank(alice);
    vault.instantRedeem(sharesToRedeem, alice, 0);

    uint256 rateAfter = core.exchangeRate();
    uint256 totalAssetsAfter = core.totalAssets();
    uint256 totalSupplyAfter = stAztec.totalSupply();
    uint256 bufferAfter = vault.bufferedAssets();

    emit log_named_uint("rate AFTER (1e18-scaled)", rateAfter);

    uint256 rateDeltaBps = (rateAfter - rateBefore) * BP_DIVISOR / rateBefore;

    // Mechanical assertion: rate strictly increases because the fee stays in the buffer.
    assertGt(rateAfter, rateBefore, "instant-redemption fee accretes into share price");
}
```

Results:

```
rate BEFORE (1e18-scaled) : 1000000000000000000
rate AFTER (1e18-scaled)  : 1049999999999999999
```

Recommendations

Store the fee to not reflect the price instantly and release it slowly.